

Physics-Informed Machine Learning

Methodology and Practice

Shancong Mou

ISyE, UMN

Fall 2025

Table of Contents

1 Introduction and overview of the lecture

- PDEs
- Course Outline

2 PIML for Surrogate modeling

- Introduction to PDE
- Physics informed equation solver
- Physics informed operator learning I: Fourier Neural Operator
- Physics informed operator learning II: DeepOnet

- Identification of PDE governed-system
- Identify unknown boundary/initial conditions
- Identify unknown boundary/initial conditions
- Spatio-temporal modeling of PDE governed-system
- Forward and Inverse problem in a loop

World governed by PDEs

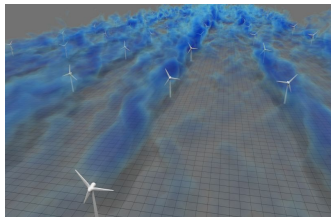


Figure: Wind farm ¹

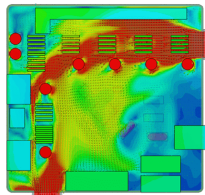


Figure: Electronic devices ²

PDE governed system

- large scale: weather system, wind farm system
- medium scale: gas turbine, 3D printing
- small scale: chips cooling, blood flow

¹(<https://pof.tnw.utwente.nl/research/turbulencebubbles/windles>)

²(<https://www.simscale.com/blog/cht-best-practices/>)

PDEs

Definition

PDE is a relation of the following type, parameterized by $\lambda \in \mathbb{R}^m$:

$$F(x_1, \dots, x_n, \dots; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2}, \dots; \lambda) = 0 \quad (1)$$

that links the variable u with respect to its partial derivatives u_{x_i} and the spatio-temporal locations $(x_1, \dots, x_n)$ (**Implicitly**).

where

- F : a general function (linear/non-linear).
- $u = u(x_1, \dots, x_n)$: the solution of the PDE, i.e., $u : \mathbb{R}^d \mapsto \mathbb{R}$;
 ($\vec{u}$ can be a vector, i.e., $\vec{u} \in \mathbb{R}^d$, here, we assume it to be a scalar for simplicity)
- $u_{x_i} = \frac{\partial u}{\partial x_i}$, $u_{x_i x_j} = \frac{\partial^2 u}{\partial x_i \partial x_j}, \dots$
- λ : the boundary/initial conditions or parameters in the PDE.

PDEs

Definition

PDE is a relation of the following type, parameterized by $\lambda \in \mathbb{R}^m$:

$$F(x_1, \dots, x_n, \dots; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2}, \dots; \lambda) = 0 \quad (1)$$

that links the variable u with respect to its partial derivatives u_{x_i} and the spatio-temporal locations $(x_1, \dots, x_n)$ (**Implicitly**).

where

- F : a general function (linear/non-linear).
- $u = u(x_1, \dots, x_n)$: the solution of the PDE, i.e., $u : \mathbb{R}^d \mapsto \mathbb{R}$;
 ($\vec{u}$ can be a vector, i.e., $\vec{u} \in \mathbb{R}^d$, here, we assume it to be a scalar for simplicity)
- $u_{x_i} = \frac{\partial u}{\partial x_i}$, $u_{x_i x_j} = \frac{\partial^2 u}{\partial x_i \partial x_j}, \dots$
- λ : the boundary/initial conditions or parameters in the PDE.

Solving a PDE:

Find a $u = u(x_1, \dots, x_n)$ function (**explicitly**) that satisfies the equation (1), i.e.,

- $u = x^2$, $u = \sin(x)$, etc...

Forward problem

Forward problem: Given a fixed λ , solve for $u(x)$

Consider the following PDE parameterized by $\lambda \in \mathbb{R}^m$:

$$F(x_1, \dots, x_n, \dots; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2}, \dots; \lambda) = 0$$

Traditionally, solved with finite difference method (FDM), finite element method (FEM), etc.

Advantages:

- Accurate and Reliable

Challenges:

- Expensive and time consuming; Less flexible (incorporate data?)

Forward problem

Forward problem: Given a fixed λ , solve for $u(x)$

Consider the following PDE parameterized by $\lambda \in \mathbb{R}^m$:

$$F(x_1, \dots, x_n, \dots; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2}, \dots; \lambda) = 0$$

Traditionally, solved with finite difference method (FDM), finite element method (FEM), etc.

Advantages:

- Accurate and Reliable

Challenges:

- Expensive and time consuming; Less flexible (incorporate data?)

Learning is suitable for:

- Surrogate modeling: find a cheap model to surrogate the PDE governed system, i.e., $u = f_\theta(x; \lambda)$.
- Incorporating physical knowledge (PDE) information for downstream tasks.

Example: PIML for multistage composite material manufacturing quality prediction

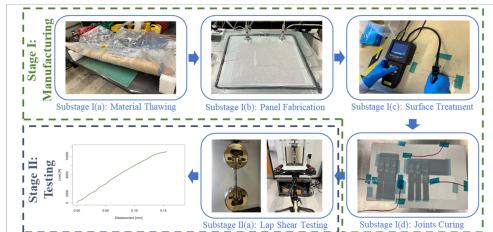


Fig. 1: Multi-stage manufacturing and testing processes of composite adhesive joints.

Example: PIML for multistage composite material manufacturing quality prediction

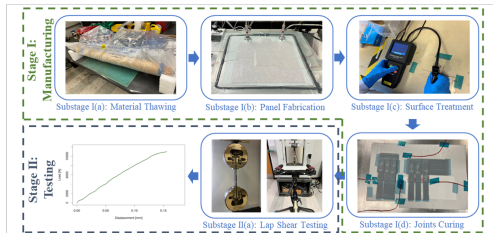


Fig. 1: Multi-stage manufacturing and testing processes of composite adhesive joints.

Problem setting:

- Task: predicting bonding quality from manufacturing process parameters
- Challenges:
 - limited training samples (expensive experiment)
 - complex model structure + heterogeneous data type
 - partially known physics in some stages
- Solution: Physics Informed Neural ODE wang2024physics

Inverse problems

Inverse problem: Given a set of observed $u(x)$, find λ

Consider the following PDE parameterized by $\lambda \in \mathbb{R}^m$:

$$F(x_1, \dots, x_n, \dots; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2}, \dots; \lambda) = 0$$

Traditionally, formulated as a PDE constraint optimization problem, and solved with adjoint method.

Learning is suitable for:

- Incorporating PDE information in inverse problems
- Surrogate modeling 1: find a cheap model to surrogate the PDE governed system, i.e., $u = f_\theta(x; \lambda)$.
- Surrogate modeling 2: find a cheap model to surrogate the inversion of PDE governed system, i.e., $\lambda = f_{\theta'}(u)$.

Inverse problems

Inverse problem: Given a set of observed $u(x)$, find λ

Consider the following PDE parameterized by $\lambda \in \mathbb{R}^m$:

$$F(x_1, \dots, x_n, \dots; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2}, \dots; \lambda) = 0$$

Traditionally, formulated as a PDE constraint optimization problem, and solved with adjoint method.

Learning is suitable for:

- Incorporating PDE information in inverse problems
- Surrogate modeling 1: find a cheap model to surrogate the PDE governed system, i.e., $u = f_\theta(x; \lambda)$.
- Surrogate modeling 2: find a cheap model to surrogate the inversion of PDE governed system, i.e., $\lambda = f_{\theta'}(u)$.

Popular applications

- Identifying unknown parameters in PDEs/boundary/initial conditions
- Data driven (with partial physics knowledge) spatio-temporal modeling

Example: Non-destructive testing using full waveform inversion

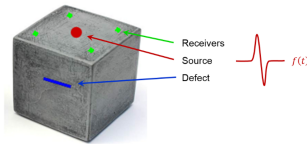


Figure: Problem setting ³

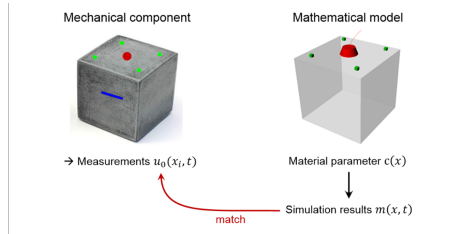


Figure: The inversion process

Example: Non-destructive testing using full waveform inversion

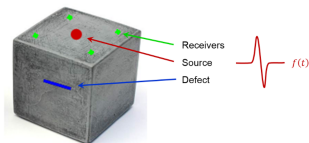


Figure: Problem setting³

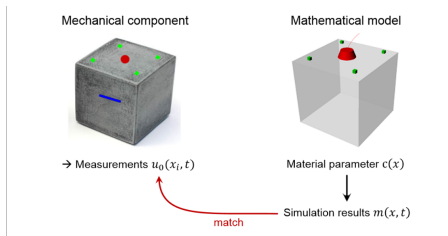


Figure: The inversion process

Problem setting:

- PDE: $F(u(x, t), f(t), c(x)) = 0$
- Task: infer material property $c(x)$ from measurement $u(x_i, t)$ on locations x_i .
- **Solution:** inverse surrogate modeling

³(<https://www.cie.bgu.tum.de/en/17-forschung/projekte/292-ndt-fwi.html>)

Course outline

- PIML as a surrogate model for PDE governed-system
 - PIML as an PDE solver (single instance)
 - PIML for operator learning (multiple instance)

Course outline

- PIML as a surrogate model for PDE governed-system
 - PIML as an PDE solver (single instance)
 - PIML for operator learning (multiple instance)
- PIML for inverse problem
 - Identifying unknown parameters/ unknown governing equations
 - Spatio-temporal modeling of PDE governed system

Course Format and Goal

For each approach, we mainly focus on

- Representative literature discussion
 - Motivation behind the proposed approach
 - Connection with current numerical methods
(will go through simple numerical algorithms if necessary)
 - Design considerations
 - Representative applications

Course Format and Goal

For each approach, we mainly focus on

- Representative literature discussion
 - Motivation behind the proposed approach
 - Connection with current numerical methods
(will go through simple numerical algorithms if necessary)
 - Design considerations
 - Representative applications
- Point out publicly available datasets/ packages

Course Format and Goal

For each approach, we mainly focus on

- Representative literature discussion
 - Motivation behind the proposed approach
 - Connection with current numerical methods
(will go through simple numerical algorithms if necessary)
 - Design considerations
 - Representative applications
- Point out publicly available datasets/ packages

Take home message:

- What can be done in the field of PIML... (for PIML research)
- What can be borrowed from PIML and contribute to your research: A way of learning and representing a PDE governed physical system

Table of Contents

- 1 Introduction and overview of the lecture
 - PDEs
 - Course Outline
- 2 PIML for Surrogate modeling
 - Introduction to PDE
 - Physics informed equation solver
 - Physics informed operator learning I: Fourier Neural Operator
 - Physics informed operator learning II: DeepOnet
- 3 PIML for inverse problem
 - Identification of PDE governed-system
 - Identify unknown boundary/initial conditions
 - Identify unknown boundary/initial conditions
 - Spatio-temporal modeling of PDE governed-system
 - Forward and Inverse problem in a loop
- 4 Summary

Introduction to PDE

Definition

Partial differential equations

Consider the PDE parameterized by $\lambda \in \mathbb{R}^m$ for the solution $u(\vec{x})$ with $\vec{x} = (x_1, \dots, x_n)$

$$f(\vec{x}; u; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2}, \dots; \lambda) = 0, \quad \vec{x} \in \Omega, \quad (2)$$

with suitable boundary conditions

$$\mathcal{B}(u, \vec{x}) = 0, \quad \vec{x} \in \partial\Omega, \quad (3)$$

where

- $u = u(x_1, \dots, x_n)$ is a unknown function of n variables, i.e., $u : \mathbb{R}^d \mapsto \mathbb{R}$;
 ($\vec{u}$ can be a vector, i.e., $\vec{u} \in \mathbb{R}^d$, here, we assume it to be a scalar for simplicity)
- $u_{x_i} = \frac{\partial u}{\partial x_i}, u_{x_i x_j} = \frac{\partial^2 u}{\partial x_i \partial x_j}, \dots$
- f is the governing equation.
- $\mathcal{B}$ is the boundary condition.

Governing equation: Linear vs. Non-linear

Linear PDE

A PDE is linear if and only if both f is linear with respect to u and all their derivatives.

$$f(\vec{x}; u; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2} \dots; \lambda) = 0, \quad \vec{x} \in \Omega, \quad (4)$$

Governing equation: Linear vs. Non-linear

Linear PDE

A PDE is linear if and only if both f is linear with respect to u and all their derivatives.

$$f(\vec{x}; u; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2} \dots; \lambda) = 0, \quad \vec{x} \in \Omega, \quad (4)$$

Non-linear PDE

- **Semilinear PDE** where f is nonlinear only with respect to u but is linear with respect to all its derivatives;
- **Quasi-linear PDE** where f is linear with respect to the highest order derivatives of u ;
- **Fully nonlinear PDE** where f is nonlinear with respect to the highest order derivatives of u .

Examples I: Burgers equation

Non-linear PDE

- **Semilinear PDE** where f is nonlinear only with respect to u but is linear with respect to all its derivatives;
- **Quasi-linear PDE** where f is linear with respect to the highest order derivatives of u ;
- **Fully nonlinear PDE** where f is nonlinear with respect to the highest order derivatives of u .

viscous Burgers' equation

A fundamental partial differential equation and convection–diffusion equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

where ν is the kinematic viscosity.

Examples I: Burgers equation

Non-linear PDE

- **Semilinear PDE** where f is nonlinear only with respect to u but is linear with respect to all its derivatives;
- **Quasi-linear PDE** where f is linear with respect to the highest order derivatives of u ;
- **Fully nonlinear PDE** where f is nonlinear with respect to the highest order derivatives of u .

viscous Burgers' equation

A fundamental partial differential equation and convection–diffusion equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

where ν is the kinematic viscosity.

- Quasi-linear PDE

Governing equation: order of PDEs

Order of a PDE

The highest order of differentiation occurring in the equation is the order of the equation.

$$f(\vec{x}; u; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2} \dots; \lambda) = 0, \quad \vec{x} \in \Omega, \quad (5)$$

Governing equation: order of PDEs

Order of a PDE

The highest order of differentiation occurring in the equation is the order of the equation.

$$f(\vec{x}; u; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2} \dots; \lambda) = 0, \quad \vec{x} \in \Omega, \quad (5)$$

Second order PDEs

Most commonly used in engineering applications.

$$au_{xx} + 2bu_{xy} + cu_{yy} + du_x + eu_y + hu = f$$

where $a, \dots, f$ are smooth (e.g. C^2) functions of x, y .

Governing equation: order of PDEs

Order of a PDE

The highest order of differentiation occurring in the equation is the order of the equation.

$$f(\vec{x}; u; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2} \dots; \lambda) = 0, \quad \vec{x} \in \Omega, \quad (5)$$

Second order PDEs

Most commonly used in engineering applications.

$$au_{xx} + 2bu_{xy} + cu_{yy} + du_x + eu_y + hu = f$$

where $a, \dots, f$ are smooth (e.g. C^2) functions of x, y .

- **Elliptic:** $b^2 - ac < 0$, e.g., Laplace equation $u_{xx} + u_{yy} = 0$

Governing equation: order of PDEs

Order of a PDE

The highest order of differentiation occurring in the equation is the order of the equation.

$$f(\vec{x}; u; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2} \dots; \lambda) = 0, \quad \vec{x} \in \Omega, \quad (5)$$

Second order PDEs

Most commonly used in engineering applications.

$$au_{xx} + 2bu_{xy} + cu_{yy} + du_x + eu_y + hu = f$$

where $a, \dots, f$ are smooth (e.g. C^2) functions of x, y .

- **Elliptic:** $b^2 - ac < 0$, e.g., Laplace equation $u_{xx} + u_{yy} = 0$
- **Parabolic:** $b^2 - ac = 0$, e.g., Diffusion equation $u_t - Du_{xx} = 0$

Governing equation: order of PDEs

Order of a PDE

The highest order of differentiation occurring in the equation is the order of the equation.

$$f(\vec{x}; u; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2} \dots; \lambda) = 0, \quad \vec{x} \in \Omega, \quad (5)$$

Second order PDEs

Most commonly used in engineering applications.

$$au_{xx} + 2bu_{xy} + cu_{yy} + du_x + eu_y + hu = f$$

where $a, \dots, f$ are smooth (e.g. C^2) functions of x, y .

- **Elliptic:** $b^2 - ac < 0$, e.g., Laplace equation $u_{xx} + u_{yy} = 0$
- **Parabolic:** $b^2 - ac = 0$, e.g., Diffusion equation $u_t - Du_{xx} = 0$
- **Hyperbolic:** $b^2 - ac > 0$, e.g., Wave equation $u_{tt} - c^2 u_{xx} = 0$

Examples I: Burgers equation

Second order PDEs

Most commonly used in engineering applications.

$$au_{xx} + 2bu_{xy} + cu_{yy} + du_x + eu_y + hu = f$$

where $a, \dots, f$ are smooth (e.g. C^2) functions of x, y .

- **Elliptic:** $b^2 - ac < 0$, e.g., Laplace equation $u_{xx} + u_{yy} = 0$
- **Parabolic:** $b^2 - ac = 0$, e.g., Diffusion equation $u_t - Du_{xx} = 0$
- **Hyperbolic:** $b^2 - ac > 0$, e.g., Wave equation $u_{tt} - c^2 u_{xx} = 0$

viscous Burgers' equation

A fundamental partial differential equation and convection–diffusion equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

where ν is the kinematic viscosity.

Examples I: Burgers equation

Second order PDEs

Most commonly used in engineering applications.

$$au_{xx} + 2bu_{xy} + cu_{yy} + du_x + eu_y + hu = f$$

where $a, \dots, f$ are smooth (e.g. C^2) functions of x, y .

- **Elliptic:** $b^2 - ac < 0$, e.g., Laplace equation $u_{xx} + u_{yy} = 0$
- **Parabolic:** $b^2 - ac = 0$, e.g., Diffusion equation $u_t - Du_{xx} = 0$
- **Hyperbolic:** $b^2 - ac > 0$, e.g., Wave equation $u_{tt} - c^2 u_{xx} = 0$

viscous Burgers' equation

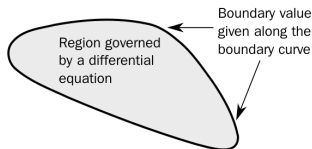
A fundamental partial differential equation and convection–diffusion equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}$$

where ν is the kinematic viscosity.

- Second order Parabolic

Boundary conditions: Types of boundary value problems



- Dirichlet: specifies the boundary value of u : $u|_{\partial\Omega} = f$

Figure: Boundary value problem ⁴

⁴https://en.wikipedia.org/wiki/Boundary_value_problem

Boundary conditions: Types of boundary value problems

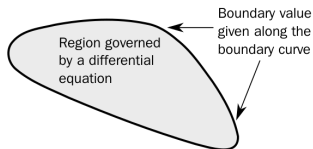


Figure: Boundary value problem ⁴

- Dirichlet: specifies the boundary value of u : $u|_{\partial\Omega} = f$
- Neumann: specifies the value of the normal derivative of the u : $u_x|_{\partial\Omega} = f$

⁴https://en.wikipedia.org/wiki/Boundary_value_problem

Boundary conditions: Types of boundary value problems

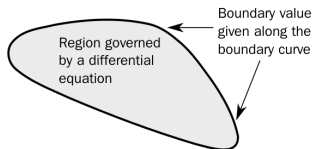


Figure: Boundary value problem ⁴

- Dirichlet: specifies the boundary value of u : $u|_{\partial\Omega} = f$
- Neumann: specifies the value of the normal derivative of the u : $u_x|_{\partial\Omega} = f$
- Robin: $c_0 u|_{\partial\Omega} + c_1 u_x|_{\partial\Omega} = f$

⁴https://en.wikipedia.org/wiki/Boundary_value_problem

Boundary conditions: Types of boundary value problems

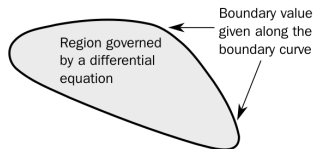


Figure: Boundary value problem ⁴

- Dirichlet: specifies the boundary value of u : $u|_{\partial\Omega} = f$
- Neumann: specifies the value of the normal derivative of the u : $u_x|_{\partial\Omega} = f$
- Robin: $c_0 u|_{\partial\Omega} + c_1 u_x|_{\partial\Omega} = f$
- Cauchy: Dirichlet and Neumann, i.e., $u|_{\partial\Omega} = f$ and $u_x|_{\partial\Omega} = f$

⁴https://en.wikipedia.org/wiki/Boundary_value_problem

Boundary conditions: Types of boundary value problems

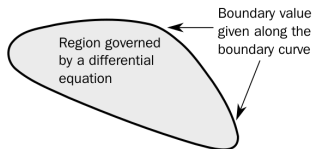


Figure: Boundary value problem ⁴

- Dirichlet: specifies the boundary value of u : $u|_{\partial\Omega} = f$
- Neumann: specifies the value of the normal derivative of the u : $u_x|_{\partial\Omega} = f$
- Robin: $c_0 u|_{\partial\Omega} + c_1 u_x|_{\partial\Omega} = f$
- Cauchy: Dirichlet and Neumann, i.e., $u|_{\partial\Omega} = f$ and $u_x|_{\partial\Omega} = f$
- Mixed: different location (x) have different boundary condition.

⁴https://en.wikipedia.org/wiki/Boundary_value_problem

Solving a Partial differential equation

Solving a Partial differential equation

Find a solution u that satisfies the governing equation f and also satisfies the boundary conditions $\mathcal{B}$.

$$f(\vec{x}; u; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2} \dots; \lambda) = 0, \quad \vec{x} \in \Omega, \quad (6)$$

with suitable boundary conditions

$$\mathcal{B}(u, \vec{x}) = 0, \quad \vec{x} \in \partial\Omega, \quad (7)$$

Solving a Partial differential equation

Solving a Partial differential equation

Find a solution u that satisfies the governing equation f and also satisfies the boundary conditions $\mathcal{B}$.

$$f(\vec{x}; u; u_{x_1}, \dots, u_{x_n}; u_{x_1 x_1}, u_{x_1 x_2} \dots; \lambda) = 0, \quad \vec{x} \in \Omega, \quad (6)$$

with suitable boundary conditions

$$\mathcal{B}(u, \vec{x}) = 0, \quad \vec{x} \in \partial\Omega, \quad (7)$$

Traditionally,

- Analytically: only works for simple PDEs with regular boundary conditions
- Numerically
 - Finite difference method (FDM)
 - Finite element method (FEM), etc.

PDE Examples I: Burgers equation

1-dimensional viscous Burgers' equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in [-1, 1], \quad t \in [0, 1]$$

$$IC : u(x, 0) = -\sin(\pi * x)$$

$$BC : u(-1, t) = u(1, t) = 0$$

where ν is the kinematic viscosity.

PDE Examples I: Burgers equation

1-dimensional viscous Burgers' equation

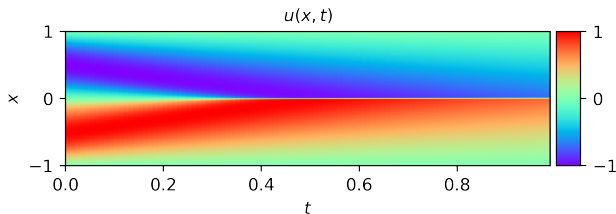
$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in [-1, 1], \quad t \in [0, 1]$$

$$IC : u(x, 0) = -\sin(\pi * x)$$

$$BC : u(-1, t) = u(1, t) = 0$$

where ν is the kinematic viscosity.

- Solution



PDE Examples II: steady state Darcy's flow equation (heat transfer)

steady-state of the 2-d Darcy Flow equation on the unit box

A fundamental partial differential equation modeling the pressure of subsurface flow, the deformation of linearly elastic materials, and the heat transfer.

$$\begin{aligned} -\nabla \cdot (a(x) \nabla u(x)) &= f(x), \quad x \in (0, 1)^2 \\ u(x) &= 0, \quad x \in \partial(0, 1)^2 \end{aligned}$$

where

- $a(x)$ is the heterogeneous permeability (thermal conductivity)
- $u(x)$ is the solution pressure of subsurface flow (temperature)
- $f(x)$ is the source term (heat source)

PDE Examples II: steady state Darcy's flow equation (heat transfer)

steady-state of the 2-d Darcy Flow equation on the unit box

A fundamental partial differential equation modeling the pressure of subsurface flow, the deformation of linearly elastic materials, and the heat transfer.

$$\begin{aligned} -\nabla \cdot (a(x) \nabla u(x)) &= f(x), \quad x \in (0, 1)^2 \\ u(x) &= 0, \quad x \in \partial(0, 1)^2 \end{aligned}$$

where

- $a(x)$ is the heterogeneous permeability (thermal conductivity)
 - $u(x)$ is the solution pressure of subsurface flow (temperature)
 - $f(x)$ is the source term (heat source)
-
- Linear PDE
 - Second order elliptic

Physics informed equation solver

Physics informed loss function (Raissi, Perdikaris, and Karniadakis 2019)

Consider the PDE parameterized by λ

$$u_t - \mathcal{L}[u; \lambda] = 0, \quad t, x \in [0, T] \times \Omega \quad (8)$$

$$\mathcal{B}(u, \vec{x}) = 0, \quad \vec{x} \in \partial\Omega,$$

where $u(x, t)$ denotes the solution, $\mathcal{L}[\cdot; \lambda]$ is a nonlinear operator.

Example: 1-dimensional viscous Burgers' equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in [-1, 1], \quad t \in [0, 1]$$

$$IC : u(x, 0) = -\sin(\pi * x)$$

$$BC : u(-1, t) = u(1, t) = 0$$

where ν is the kinematic viscosity.

Physics informed loss function (Raissi, Perdikaris, and Karniadakis 2019)

Consider the PDE parameterized by λ

$$u_t - \mathcal{L}[u; \lambda] = 0, \quad t, x \in [0, T] \times \Omega \quad (8)$$

$$\mathcal{B}(u, \vec{x}) = 0, \quad \vec{x} \in \partial\Omega,$$

where $u(x, t)$ denotes the solution, $\mathcal{L}[\cdot; \lambda]$ is a nonlinear operator.

Example: 1-dimensional viscous Burgers' equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in [-1, 1], \quad t \in [0, 1]$$

$$IC : u(x, 0) = -\sin(\pi * x)$$

$$BC : u(-1, t) = u(1, t) = 0$$

where ν is the kinematic viscosity.

Solving PDE "means" find a solution $u(x, t)$ satisfying the governing equation and boundary conditions, **for all $x, t \in [0, T] \times \Omega$.**

Physics informed loss function (Raissi, Perdikaris, and Karniadakis 2019)

Consider the PDE parameterized by λ

$$u_t - \mathcal{L}[u; \lambda] = 0, \quad t, x \in [0, T] \times \Omega \quad (9)$$

$$\mathcal{B}(u, \vec{x}) = 0, \quad \vec{x} \in \partial\Omega, \quad (10)$$

where $u(x, t)$ denotes the solution, $\mathcal{L}[\cdot; \lambda]$ is a nonlinear operator.

Solving PDE "means" find a solution $u(x, t)$ satisfying the governing equation and boundary conditions, for all $x, t \in [0, T] \times \Omega$.

Use parameterized function $h(x, t; \theta)$ to approximate $u(x, t)$

Find a $h(x, t; \theta) \approx u(x, t)$, such that

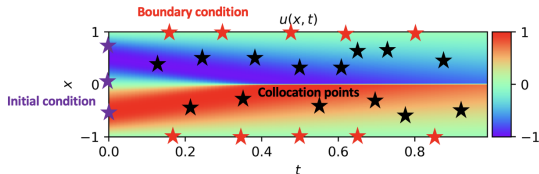
$$h_t - \mathcal{L}[h; \lambda] \approx 0, \quad \forall t, x \in [0, T] \times \Omega$$

$$\mathcal{B}(h, \vec{x}) \approx 0, \quad \vec{x} \in \partial\Omega,$$

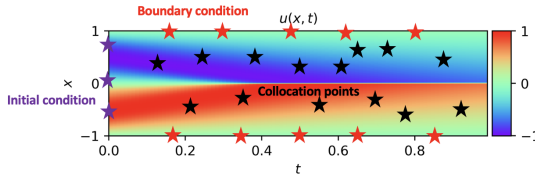
How to formulate this as a learning problem?

Main idea: solving PDE $\rightarrow$ training neural network $h(x, t; \theta)$

Minimizing the 'violation' of the governing equation/boundary/initial condition at multiple collocations/boundary/initial points: $(x_i, t_j), i \in I, j \in J$.



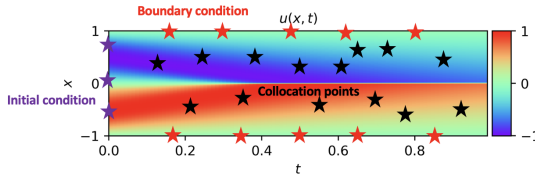
How to formulate this as a learning problem?



Loss function?

$$\begin{aligned} \min_{\theta} \quad & \sum_{i \in I_{coll}, j \in J_{coll}} \|h_t(x_i, t_j; \theta) - \mathcal{L}(h(x_i, t_j; \theta); \lambda)\|_2^2 + \sum_{i \in I_{ini}} \|h_t(x_i, t_0) - \mathcal{L}(h(x_i, t_0); \lambda)\|_2^2 \\ & + \sum_{j \in J_{bnd}} \|h_t(x_0, t_j) - \mathcal{L}(h(x_0, t_j); \lambda)\|_2^2 \end{aligned}$$

How to formulate this as a learning problem?



Loss function?

$$\begin{aligned} \min_{\theta} \quad & \sum_{i \in I_{coll}, j \in J_{coll}} \|h_t(x_i, t_j; \theta) - \mathcal{L}(h(x_i, t_j; \theta); \lambda)\|_2^2 + \sum_{i \in I_{ini}} \|h_t(x_i, t_0) - \mathcal{L}(h(x_i, t_0); \lambda)\|_2^2 \\ & + \sum_{j \in J_{bnd}} \|h_t(x_0, t_j) - \mathcal{L}(h(x_0, t_j); \lambda)\|_2^2 \end{aligned}$$

- All the partial derivatives can be calculated by 'automatic differentiation'.
- Modern gradient based optimizer can be directly applied.

Burger's equation as an example 1: define a fully connected network $h(x, t)$

Burger's equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in [-1, 1], \quad t \in [0, 1]$$

$$IC : u(x, 0) = -\sin(\pi * x)$$

$$BC : u(-1, t) = u(1, t) = 0$$

Neural network $h(x, t)$ as an approximation of u

- Network input: coordinate values $(x, t) \in \mathbb{R}^2$;
- Network output: predicted solution value at (x, t) , i.e., $u(x, t) \in \mathbb{R}$

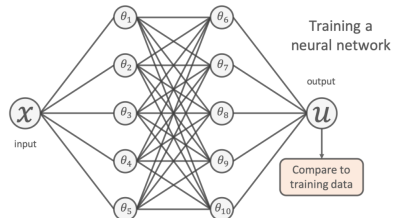


Figure: A fully connected neural network h

Coding example: define a fully connected network $h(x, t)$

```
'Initialise neural network as a list using nn.ModuleList'
self.linears = nn.ModuleList([nn.Linear(layers[i], layers[i+1]) for i in range(len(layers)-1)])
self.iter = 0
# std = gain * sqrt(2/(input_dim+output_dim))
for i in range(len(layers)-1):
    # weights from a normal distribution with
    nn.init.xavier_normal_(self.linears[i].weight.data, gain=1.0)
    # set biases to zero
    nn.init.zeros_(self.linears[i].bias.data)
'foward pass'
def h(self,x):
    if torch.is_tensor(x) != True:
        x = torch.from_numpy(x)
    u_b = torch.from_numpy(ub).float().to(device)
    l_b = torch.from_numpy(lb).float().to(device)
    #preprocessing input
    x = (x - l_b)/(u_b - l_b) #feature scaling
    #convert to float
    a = x.float()
    for i in range(len(layers)-2):
        z = self.linears[i](a)
        a = self.activation(z)
    u = self.linears[-1](a)
    return u
```

Burger's equation as an example 1: define losses functions

Burger's equation

$$\frac{\partial h}{\partial t} + h \frac{\partial h}{\partial x} = \nu \frac{\partial^2 h}{\partial x^2}, \quad x \in [-1, 1], \quad t \in [0, 1]$$

$$IC : h(x, 0) = -\sin(\pi * x)$$

$$BC : h(-1, t) = h(-1, t)(1, t) = 0$$

Losses to evaluate approximation result $h(x, t) \approx u$

- Violation of the governing equation:

$$\left\| \frac{\partial h}{\partial t} + h \frac{\partial h}{\partial x} - \nu \frac{\partial^2 h}{\partial x^2} \right\|_2^2$$

- Violation of the governing equation boundary/initial conditions

$$\|h(x, 0) + \sin(\pi * x)\|_2^2, \|h(\pm 1, t) - 0\|_2^2$$

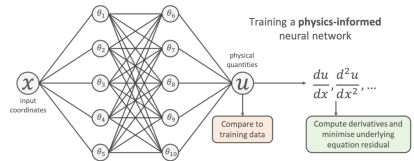


Figure: A fully connected neural network h

^a<https://benmoseley.blog/my-research/so-what-is-a-physics-informed-neural-network/>

Coding example: Losses to evaluate approximation result

```
def loss_BC(self,x,y):
    loss_u = self.loss_function(self.h(x), y)
    return loss_u

def loss_PDE(self, x_collocation):
    nu = 0.01/np.pi
    g = x_collocation.clone()
    g.requires_grad = True
    u = self.h(g)
    u_x_t = autograd.grad(u,g,torch.ones([x_collocation.shape[0], 1]).to(device),
                           retain_graph=True, create_graph=True)[0]

    u_x = u_x_t[:,[0]]
    u_t = u_x_t[:,[1]]
    u_xx = u_xx_tt[:,[0]]
    f = u_t + (self.h(g))*(u_x) - (nu)*u_xx
    loss_f = self.loss_function(f, f_hat)
    return loss_f

def loss(self,x,y,x_collocation):

    loss_u = self.loss_BC(x,y)
    loss_f = self.loss_PDE(x_collocation)

    loss_val = loss_u + loss_f
```

Optimization

```
'Generate Training data'
N_u = 100 #Total number of data points for 'u'
N_f = 10_000 #Total number of collocation points
x_collocation_np_array, X_u_train_np_array, u_train_np_array = trainingdata(N_u, N_f)
'Convert to tensor and send to GPU'
x_collocation = torch.from_numpy(x_collocation_np_array).float().to(device)
X_u_train = torch.from_numpy(X_u_train_np_array).float().to(device)
u_train = torch.from_numpy(u_train_np_array).float().to(device)
X_u_test_tensor = torch.from_numpy(X_u_test).float().to(device)
u = torch.from_numpy(u_true).float().to(device)
f_hat = torch.zeros(x_collocation.shape[0], 1).to(device)
layers = nn.array([2, 20, 20, 20, 20, 20, 20, 20, 20, 1]) #8 hidden layers
'Define PINN'
PINN = SequentialModel(layers)
PINN.to(device)
'Optimization: Adam Optimizer'
optimizer = optim.Adam(PINN.parameters(), lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0, amsgrad=False)
max_iter = 20000
start_time = time.time()
for i in range(max_iter):
    loss = PINN.loss(X_u_train, u_train, x_collocation)
    optimizer.zero_grad() # zeroes the gradient buffers of all parameters
    loss.backward() #backprop
    optimizer.step()
    if i % (max_iter/100) == 0:
        error_vec, _ = PINN.test()
        print(loss, error_vec)
```

Take home message

Pros and Cons

Pros:

- Easy and effective way of incorporating PDE governed process knowledge into your problem as **an additional loss term**. For example, your process is controlled by a PDE and you also collected several measurements $u_{obs}(x'_i, t'_j), i \in I', j' \in J'$

$$\hat{\theta} = \arg \min_{\theta} \underbrace{\sum_{i \in I', j \in J'} \|u_{obs}(x'_i, t'_j) - h_t(x'_i, t'_j)\|}_{data \text{ loss}} + \underbrace{\sum_{i \in I, j \in J} \|h_t(x_i, t_i) - \mathcal{L}(h(x_i, t_i); \lambda)\|}_{knowledge \text{ loss}}$$

- Directly incorporate the Forward problem with modern Deep Learning framework (i.e., Pytorch, Tensorflow, etc)

Take home message

Pros and Cons

Pros:

- Easy and effective way of incorporating PDE governed process knowledge into your problem as **an additional loss term**. For example, your process is controlled by a PDE and you also collected several measurements $u_{obs}(x'_i, t'_j), i \in I', j' \in J'$

$$\hat{\theta} = \arg \min_{\theta} \underbrace{\sum_{i \in I', j \in J'} \|u_{obs}(x'_i, t'_j) - h_t(x'_i, t'_j)\|}_{data \text{ loss}} + \underbrace{\sum_{i \in I, j \in J} \|h_t(x_i, t_i) - \mathcal{L}(h(x_i, t_i); \lambda)\|}_{knowledge \text{ loss}}$$

- Directly incorporate the Forward problem with modern Deep Learning framework (i.e., Pytorch, Tensorflow, etc)

Cons

- Accuracy guarantee?
- For a fixed λ ?
- Optimization problem difficult to converge?

Connection to FEM: Weak form⁶

Weak form: minimizing the residual

Define the 'Residual': $R(x, t) = \|u_t - \mathcal{L}[u; \lambda]\|$. Then the weak form is defined as:

$$\int_0^T \int_{\Omega} R(x, t) v(x, t) = 0, \quad t, x \in [0, T] \times \Omega$$

Aim: find a function $h(x, t)$ can minimize such weighted residual, then $h(x, t) \approx u(x, t)$ as a weak solution ^a.

^aUsually assume $h(x, t)$ is represented under a set of basis with unknown parameter $c_l(x, t)$, i.e., $h(x, t) = \sum_l c_l(x, t) \phi_l(x, t)$

- Collocation method: $v(x, t) = \delta(x_i, t_j), i \in I, j \in J$

$$\int_0^T \int_{\Omega} R(x, t) v(x, t) = R(x_i, t_j) = 0, i \in I, j \in J$$

PINN can be interpreted as a collocation method by using NN as the "basis function".

⁵Dirac delta function

⁶Some good explanations: The Finite Element Method (FEM); Finite elements/Weighted residual methods

Physics informed operator learning I: Fourier Neural Operator

Operator

Example: Darcy flow problem with varying diffusion coefficient

Given $a(x)$, solve for $u(x)$.

$$\begin{aligned} -\nabla \cdot (a(x) \nabla u(x)) &= f(x), \quad x \in (0, 1)^2 \\ u(x) &= 0, \quad x \in \partial(0, 1)^2 \end{aligned}$$

Operator

Example: Darcy flow problem with varying diffusion coefficient

Given $a(x)$, solve for $u(x)$.

$$-\nabla \cdot (a(x) \nabla u(x)) = f(x), \quad x \in (0, 1)^2$$

$$u(x) = 0, \quad x \in \partial(0, 1)^2$$

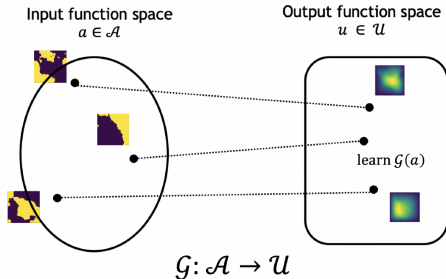


Figure: Neural operator for Darcy flow Li et al. 2020a

Operator

Consider the following PDE

$$\begin{aligned}\mathcal{F}(u, a) &= 0, \text{ in } D \subset \mathcal{R}^d \\ u &= g, \text{ in } \partial D\end{aligned}$$

- $a \in \mathcal{A}$: PDE coefficient/parameter (can be locational/spatial dependent)
- $u \in \mathcal{U}$: the solution of PDE.

For each given a , solving the PDE will give us a corresponding u . There exists a functional mapping $\mathcal{G}^* : \mathcal{A} \rightarrow \mathcal{U}$.

Operator

Consider the following PDE

$$\begin{aligned}\mathcal{F}(u, a) &= 0, \text{ in } D \subset \mathcal{R}^d \\ u &= g, \text{ in } \partial D\end{aligned}$$

- $a \in \mathcal{A}$: PDE coefficient/parameter (can be locational/spatial dependent)
- $u \in \mathcal{U}$: the solution of PDE.

For each given a , solving the PDE will give us a corresponding u . There exists a functional mapping $\mathcal{G}^* : \mathcal{A} \rightarrow \mathcal{U}$.

Operator learning

We aim to learn a functional mapping from $\mathcal{G}_\theta : \mathcal{A} \rightarrow \mathcal{U}$, parameterized by $\theta \in \Theta$, such that

$$\min_{\theta \in \Theta} \mathbb{E}_{a \sim \mu} [\mathcal{L}(\mathcal{G}_\theta(a), \mathcal{G}^*(a))] \quad (11)$$

where

- $\mathcal{L} : \mathcal{U} \times \mathcal{U} \rightarrow \mathbb{R}$ is a cost function (i.e., L_2 loss)
- μ : the distribution of a .

Two different approaches

Two families of methods for operator learning

- Fourier Neural Operator (Li et al. 2020b; Li et al. 2020a)
- DeepOnet (Lu, Jin, and Karniadakis 2019; Wang, Wang, and Perdikaris 2021)

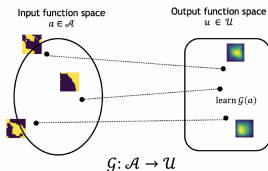
Fourier Neural Operator: Motivation problem

Darcy flow:

$$-\nabla \cdot (a(x) \nabla u(x)) = f(x), \quad x \in (0, 1)^2$$

$$u(x) = 0, \quad x \in \partial(0, 1)^2$$

Given diffusion coefficient field $a(x)$, solve for $u(x)$.

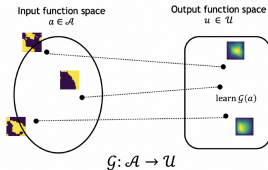


Goal: Aim to learn a mapping

$$u(x) \approx G(a(x))$$

Figure: Neural operator for Darcy flow

Fourier Neural Operator: Motivation problem



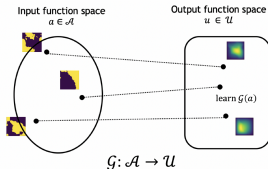
Goal: Aim to learn a mapping

$$u(x) \approx G(a(x))$$

A difficult problem (how to represent functions?)

Figure: Neural operator for Darcy flow

Fourier Neural Operator: Motivation problem



Goal: Aim to learn a mapping

$$u(x) \approx G(a(x))$$

A difficult problem (how to represent functions?)

Figure: Neural operator for Darcy flow

Discretization



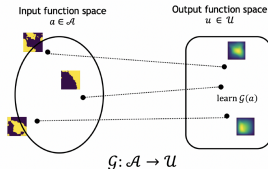
observation pairs a_j, u_j

Data: let $P_K = \{x_1, \dots, x_K\} \in D$ to be a K -point discretization of the domain D .

$$a_j = [a_j(x_1), \dots, a_j(x_K)] \in \mathbb{R}^K, j = 1, \dots, N$$

$$u_j = [u_j(x_1), \dots, u_j(x_K)] \in \mathbb{R}^K, j = 1, \dots, N$$

Fourier Neural Operator: Motivation problem



Goal: Aim to learn a mapping

$$u(x) \approx G(a(x))$$

A difficult problem (how to represent functions?)

Figure: Neural operator for Darcy flow

Discretization



Goal: Aim to learn a (discretized) mapping

$$u_j \approx G_\theta(a_j)$$

Mapping between graphs!

Graph Kernel Network: fundamental solution of a PDE

Consider the following PDE

$$\begin{aligned}\mathcal{L}_a u(x) &= f(x), & x \in D \\ u(x) &= 0, & x \in \partial D\end{aligned}$$

where

- $\mathcal{L}_a$ is a (linear) differential operator, i.e., $\mathcal{L}_a \cdot = -\operatorname{div}(a \nabla \cdot)$.

Graph Kernel Network: fundamental solution of a PDE

Consider the following PDE

$$\begin{aligned}\mathcal{L}_a u(x) &= f(x), & x \in D \\ u(x) &= 0, & x \in \partial D\end{aligned}$$

where

- $\mathcal{L}_a$ is a (linear) differential operator, i.e., $\mathcal{L}_a \cdot = -\text{div}(a \nabla \cdot)$.

Under mild conditions, we can find a solution (Green's function $G_s(x)$) for a point source s .

$$\mathcal{L}_a G_s(x) = \delta_s(x) \text{ and } G(x) = 0, \quad x \in \partial D$$

where $\delta_s(x)$ is s Dirac delta function

$$\delta_s(x) = \begin{cases} 0 & x \neq s \\ \infty & x = s \end{cases}$$

and

$$\int_D \delta_s(x) \, dx = 1 \text{ and } \int_D \delta_s(x) f(x) \, dx = f(s)$$

Graph Kernel Network: fundamental solution of a PDE

Fundamental solution of a PDE

Consider the following PDE

$$\begin{aligned}\mathcal{L}_a u(x) &= f(x), & x \in D \\ u(x) &= 0, & x \in \partial D\end{aligned}$$

if we can find a Green's function $G_s(x)$ for any point source at s , such that

$$\mathcal{L}_a G_s(x) = \delta_s(x) \text{ and } G(x) = 0, \quad x \in \partial D$$

Then, solution $u(s)$ can be written as:

$$u(s) = \int_D G_s(y) f(y) dy$$

Graph Kernel Network: fundamental solution of a PDE

Fundamental solution of a PDE

Consider the following PDE

$$\begin{aligned}\mathcal{L}_a u(x) &= f(x), & x \in D \\ u(x) &= 0, & x \in \partial D\end{aligned}$$

if we can find a Green's function $G_s(x)$ for any point source at s , such that

$$\mathcal{L}_a G_s(x) = \delta_s(x) \text{ and } G(x) = 0, \quad x \in \partial D$$

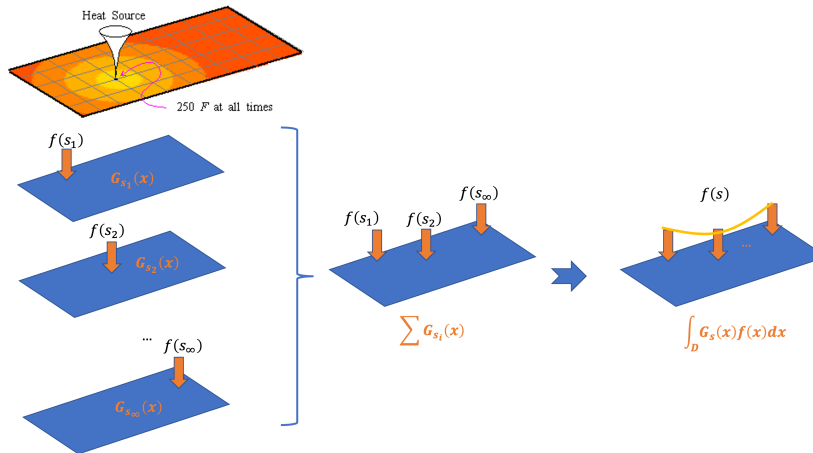
Then, solution $u(s)$ can be written as:

$$u(s) = \int_D G_s(y) f(y) dy$$

A simple proof

$$\mathcal{L}_a u(s) = \mathcal{L}_a \int_D G_s(y) f(y) dy = \int_D \mathcal{L}_a G_s(y) f(y) dy = \int_D \delta_s(y) f(y) dy = f(s)$$

Superposition



Fundamental solution

Table 1: Fundamental solutions for most commonly used operators.

Equation	One-dimensional	Two-dimensional	Three-dimensional
Laplace $\nabla^2 U = -\delta(\xi, \mathbf{x})$	$U = -\frac{ \mathbf{x} }{2}$	$U = -\frac{1}{2\pi} \ln r$	$U = -\frac{1}{4\pi r}$
Helmholtz $(\nabla^2 + \lambda^2)U = -\delta(\xi, \mathbf{x})$	$U = -\frac{1}{2\lambda} \sin(\lambda \mathbf{x})$	$U = -\frac{1}{4i} H^{(1)}(\lambda r)$	$U = -\frac{1}{4\pi r} e^{-i\lambda r}$
Modified Helmholtz $(\nabla^2 - \lambda^2)U = -\delta(\xi, \mathbf{x})$	$U = -\frac{1}{2\lambda} \sin(-i\lambda \mathbf{x})$	$U = -\frac{1}{2\pi} K_0(\lambda r)$	$U = -\frac{1}{4\pi r} e^{\lambda r}$
Bi-harmonic $\nabla^4 U = -\delta(\xi, \mathbf{x})$		$U = -\frac{1}{8\pi} r^2 \ln r$	

Graph Kernel Network: Intuition

Consider the following PDE

$$\begin{aligned}\mathcal{L}_a u(x) &= f(x), & x \in D \\ u(x) &= 0, & x \in \partial D\end{aligned}$$

where $\mathcal{L}_a$ is a (linear) differential operator, i.e., $\mathcal{L}_a \cdot = -\text{div}(a \nabla \cdot)$.

It's solution $u(x)$ can be written in convolution format

$$\begin{aligned}u(s) &= \int_D G_s^a(y) f(y) dy \\ &= \int_D G^a(s, y) f(y) dy\end{aligned}$$

Convolution inspired network design

Multiple step graph convolution:

$$v_{t+1}(x) = \sigma \left(W v_t(x) + \int_D k_\phi(x, y, a(x), a(y)) v_t(y) dy \right)$$

Graph Kernel Network

Convolution inspired network design

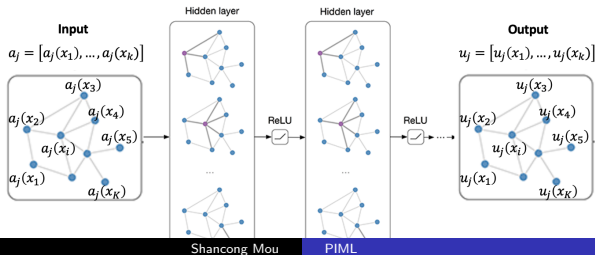
$G_\theta(a_j)$:

Input: $v_0(x) = a(x)$ *Simplified version of $P(x, a(x), a_\epsilon(x), \nabla a_\epsilon(x))$*

Hidden layers: $v_{t+1}(x) = \sigma \left(W v_t(x) + \int_D k_\phi(x, y, a(x), a(y)) v_t(y) dy \right)$

Output: $v(x) = QV^T(x) + q$

where $x \in \mathbb{R}^d$, $a(x) \in \mathbb{R}$, $v_t(x) \in \mathbb{R}^{d_v}$.



Graph Kernel Network

Use message passing graph neural network to approximate the spatial-kernel. i.e.,

$$\frac{1}{|N(x)|} \sum_{y \in N(x)} k_{\phi}(x, y, a(x), a(y)) v_t(y) \approx \int_D k_{\phi}(x, y, a(x), a(y)) v_t(y) dy$$

where $N(x)$ are collections of neighbours of x location

Graph convolution to approximate spatial-kernel

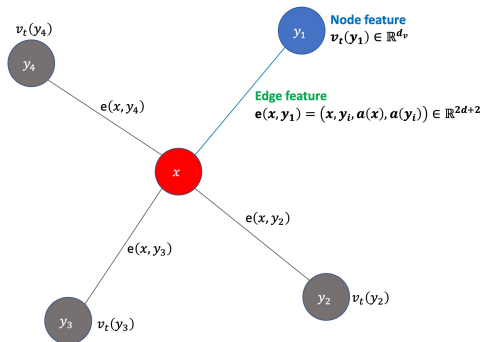
$$v_{t+1}(x) = \sigma \left(W v_t(x) + \frac{1}{|N(x)|} \sum_{y \in N(x)} k_{\phi}(x, y, a(x), a(y)) v_t(y) \right)$$

$$k_{\phi}(x, y, a(x), a(y)) : 2d + 2 \mapsto v_d \times v_d.$$

Define $e(x, y) = (x, y, a(x), a(y))$ as the edge feature with dimensionality $2d + 2$.

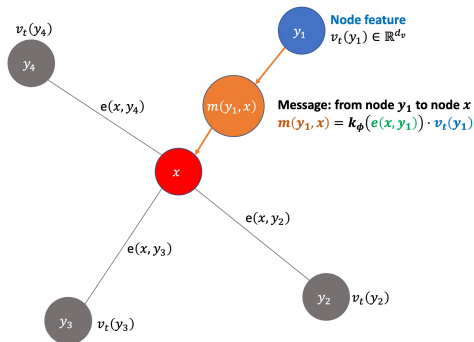
A closer look from algorithm perspective: features

$$v_{t+1}(x) = \sigma \left(Wv_t(x) + \frac{1}{|N(x)|} \sum_{y \in N(x)} k_\phi(\mathbf{e}(x, y)) v_t(y) \right)$$



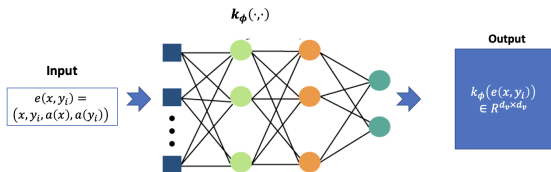
A closer look from algorithm perspective: preparing message

$$v_{t+1}(x) = \sigma \left(Wv_t(x) + \frac{1}{|N(x)|} \sum_{y \in N(x)} k_\phi(e(x, y)) v_t(y) \right)$$



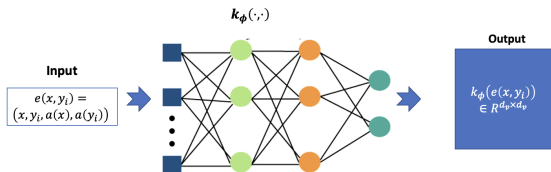
Structure of $m(y_i, x) = k_\phi(e(x, y))v_t(y_i)$

Multiplayer perceptual network with input size $2d + 2$ and output size $v_d \times v_d$. The network $k_\phi(\cdot, \cdot)$ share the same parameters at different spatio-location:



Structure of $m(y_i, x) = k_\phi(e(x, y))v_t(y_i)$

Multiplayer perceptual network with input size $2d + 2$ and output size $v_d \times v_d$. The network $k_\phi(\cdot, \cdot)$ share the same parameters at different spatio-location:

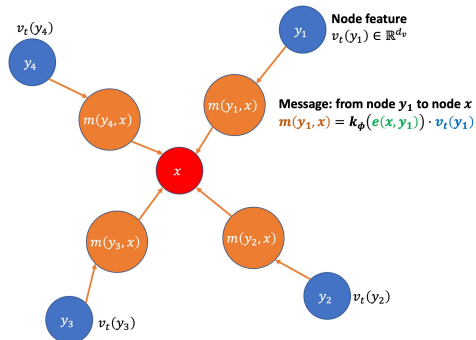


$m(y_1, x) = k_\phi(e(x, y_1))v_t(y_1)$ is a matrix and vector multiplication

$$m(y_1, x) \in \mathbb{R}^{d_v} = k_\phi(e(x, y_1)) \times v(y_1) \in \mathbb{R}^{d_v}$$

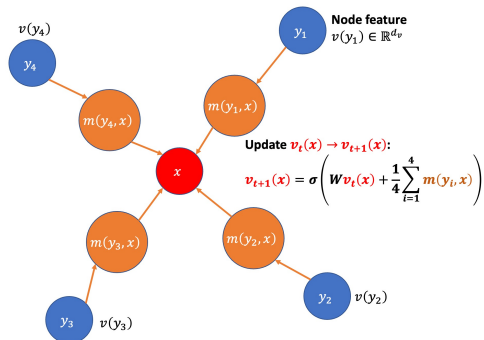
A closer look from algorithm perspective: gathering message

$$v_{t+1}(x) = \sigma \left(Wv_t(x) + \frac{1}{|N(x)|} \sum_{y \in N(x)} k_\phi(e(x, y)) v_t(y) \right)$$



A closer look from algorithm perspective: updating node x

$$v_{t+1}(x) = \sigma \left(Wv_t(x) + \frac{1}{|N(x)|} \sum_{y \in N(x)} k_\phi(e(x, y)) v_t(y) \right)$$



Loss function

observation pairs a_j, u_j

Data: let $P_K = \{x_1, \dots, x_K\} \in D$ to be a K -point discretization of the domain D .

$$a_j = [a_j(x_1), \dots, a_j(x_K)] \in \mathbb{R}^K, j = 1, \dots, N$$

$$u_j = [u_j(x_1), \dots, u_j(x_K)] \in \mathbb{R}^K, j = 1, \dots, N$$

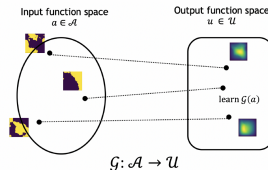


Figure: Neural operator for Darcy flow

Loss function

observation pairs a_j, u_j

Data: let $P_K = \{x_1, \dots, x_K\} \in D$ to be a K-point discretization of the domain D.

$$a_j = [a_j(x_1), \dots, a_j(x_K)] \in \mathbb{R}^K, j \in 1, \dots, N$$

$$u_j = [u_j(x_1), \dots, u_j(x_K)] \in \mathbb{R}^K, j \in 1, \dots, N$$

Loss function

$$\min_{\theta} \sum_j \|u_j - G_{\theta}(a_j)\|_2^2$$

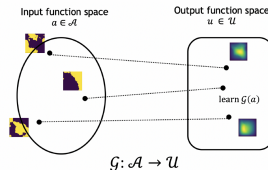
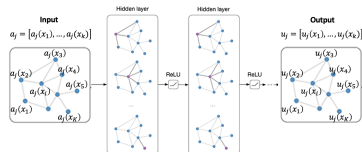


Figure: Neural operator for Darcy flow



Discussions

Advantages:

- Mesh discretization independent: networks learn the mapping between infinite-dimensional function spaces, which can then be shared with approximations at various levels of discretization.

Discussions

Advantages:

- Mesh discretization independent: networks learn the mapping between infinite-dimensional function spaces, which can then be shared with approximations at various levels of discretization.
- Mesh geometry independent due to the graph structure, can transfer between different geometries! 'For example, when computing the flow dynamic of many different airfoils, we can construct different graphs and train together.'

Discussions

Advantages:

- Mesh discretization independent: networks learn the mapping between infinite-dimensional function spaces, which can then be shared with approximations at various levels of discretization.
- Mesh geometry independent due to the graph structure, can transfer between different geometries! 'For example, when computing the flow dynamic of many different airfoils, we can construct different graphs and train together.'
- comparable performance with NN, FCN, etc.

Discussions

Advantages:

- Mesh discretization independent: networks learn the mapping between infinite-dimensional function spaces, which can then be shared with approximations at various levels of discretization.
- Mesh geometry independent due to the graph structure, can transfer between different geometries! 'For example, when computing the flow dynamic of many different airfoils, we can construct different graphs and train together.'
- comparable performance with NN, FCN, etc.

Disadvantages:

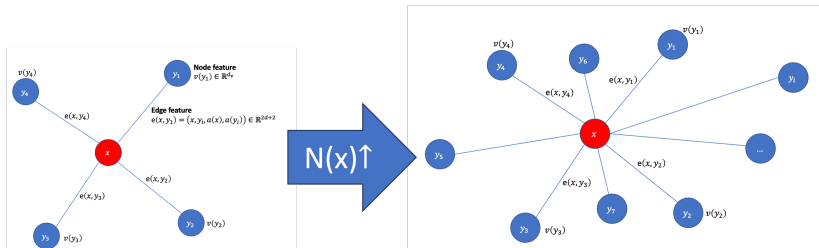
- Computation and storage scales with the number of edges (how far neighbourhood information is considered)
- Performance is sensitive to the number of edges

Fourier neural operator: how about convolution in Fourier domain?

Message passing graph convolutional layer

$$\frac{1}{|N(x)|} \sum_{y \in N(x)} k_{\phi}(e(x, y)) v_t(y) \approx \int_D k_{\phi}(x, y, a(x), a(y)) v_t(y) dy$$

$N(x)$ the larger the better (in terms of accuracy)



Fourier neural operator: how about convolution in Fourier domain?

Message passing graph convolutional layer

$$\frac{1}{|N(x)|} \sum_{y \in N(x)} k_{\phi}(e(x, y)) v_t(y) \approx \int_D k_{\phi}(x, y, a(x), a(y)) v_t(y) dy$$

$N(x)$ the larger the better (in terms of accuracy)

Large $N(x)$ lead to computational issue.

Fourier neural operator: how about convolution in Fourier domain?

Message passing graph convolutional layer

$$\frac{1}{|N(x)|} \sum_{y \in N(x)} k_{\phi}(e(x, y)) v_t(y) \approx \int_D k_{\phi}(x, y, a(x), a(y)) v_t(y) dy$$

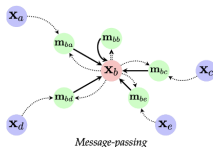
$N(x)$ the larger the better (in terms of accuracy)

Large $N(x)$ lead to computational issue.

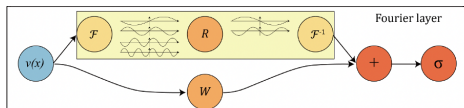
Fourier layer

Approximate the convolution in the **Fourier Space**

$$\mathcal{F}^{-1} (R_{\phi} \cdot (\mathcal{F} v_t)) (x) \approx \int_D k_{\phi}(x, y, a(x), a(y)) v_t(y) dy$$



Graph convolutional layer



Fourier layer

Fourier neural operator: preliminaries

Fourier transform

Given a real valued function $f : \mathbb{R} \mapsto \mathbb{R}$, the Fourier transformation $\hat{f} = \mathcal{F}f : \mathbb{R} \mapsto \mathbb{C}$ is defined as:

$$\hat{f}(k) = (\mathcal{F}f)(k) = \int_{-\infty}^{\infty} f(x) e^{-2\pi i x \cdot k} dx$$

^ahttps://en.wikipedia.org/wiki/Fourier_transform

Fourier neural operator: preliminaries

Fourier transform

Given a real valued function $f : \mathbb{R} \mapsto \mathbb{R}$, the Fourier transformation $\hat{f} = \mathcal{F}f : \mathbb{R} \mapsto \mathbb{C}$ is defined as:

$$\hat{f}(k) = (\mathcal{F}f)(k) = \int_{-\infty}^{\infty} f(x) e^{-2\pi i x \cdot k} dx$$

^ahttps://en.wikipedia.org/wiki/Fourier_transform

Fourier convolution theorem (convolution in real space = multiplication in Fourier space)

Define $G(f) = \mathcal{F}g(f)$ and $H(f) = \mathcal{F}h(f)$. Let $r(x) = g * h(x)$ to be the convolution, i.e.,

$$r(x) = \int_{-\infty}^{\infty} g(\tau) h(x - \tau) d\tau$$

Then

$$r(x) = \mathcal{F}^{-1}\{G \cdot H\}$$

Dimensional reduction: preliminaries

Fourier series expansion for a periodic function $f(x)$ defined in $[-T, T]$.

$$f(x) = a_0 + \sum_{n=1}^{\infty} a_n \cos\left(nx \frac{\pi}{T}\right) + \sum_{n=1}^{\infty} b_n \sin\left(nx \frac{\pi}{T}\right)$$

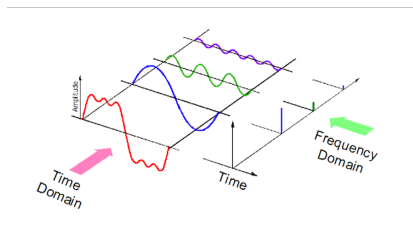


Figure: Fourier series expansion

Function approximation by first K terms

$$f(x) \approx f_K(x) = a_0 + \sum_{n=1}^K a_n \cos\left(nx \frac{\pi}{T}\right) + \sum_{n=1}^K b_n \sin\left(nx \frac{\pi}{T}\right)$$

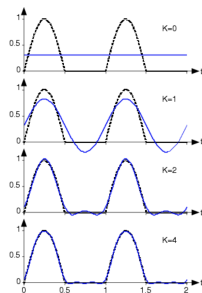


Figure: Fourier approximation

Convolution in Fourier domain

If we take $k_\phi(x, y, a(x), a(y)) = k_\phi(x - y)$, then

$$\int_D k_\phi(x, y, a(x), a(y)) v_t(y) dy = \int_D k_\phi(x - y) v_t(y) dy$$

is a convolution!

Convolution in Fourier domain

If we take $k_\phi(x, y, a(x), a(y)) = k_\phi(x - y)$, then

$$\int_D k_\phi(x, y, a(x), a(y)) v_t(y) dy = \int_D k_\phi(x - y) v_t(y) dy$$

is a convolution!

According to convolution theorem (**convolution in real space = multiplication in Fourier space**), we have

$$\int_D k_\phi(x - y) v_t(y) dy = \mathcal{F}^{-1} ((\mathcal{F} k_\phi) \cdot (\mathcal{F} v_t))$$

Convolution in Fourier domain

If we take $k_\phi(x, y, a(x), a(y)) = k_\phi(x - y)$, then

$$\int_D k_\phi(x, y, a(x), a(y)) v_t(y) dy = \int_D k_\phi(x - y) v_t(y) dy$$

is a convolution!

According to convolution theorem (**convolution in real space = multiplication in Fourier space**), we have

$$\int_D k_\phi(x - y) v_t(y) dy = \mathcal{F}^{-1} ((\mathcal{F} k_\phi) \cdot (\mathcal{F} v_t))$$

We can directly define the kernel convolution in Fourier domain, i.e. $R_\phi = \mathcal{F} k_\phi$.

$$\int_D k_\phi(x - y) v_t(y) dy = \mathcal{F}^{-1} (R_\phi \cdot (\mathcal{F} v_t))$$

Dimensional reduction in Fourier domain

- Using a maximal number of modes K_{max} to approximate $(\mathcal{F}v_t)$

$$\mathcal{F}v_t \approx (\mathcal{F}v_t)_{K_{max}}$$

Dimensional reduction in Fourier domain

- Using a maximal number of modes K_{max} to approximate $(\mathcal{F}v_t)$

$$\mathcal{F}v_t \approx (\mathcal{F}v_t)_{K_{max}}$$

- Convolution in reduced Fourier feature space

$$(R_\phi)_{K_{max}} \cdot (\mathcal{F}v_t)_{K_{max}}$$

Dimensional reduction in Fourier domain

- Using a maximal number of modes K_{max} to approximate $(\mathcal{F}v_t)$

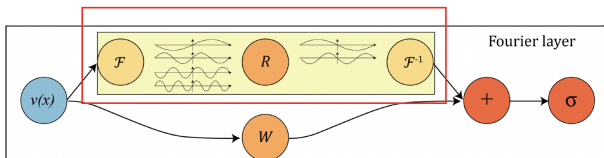
$$\mathcal{F}v_t \approx (\mathcal{F}v_t)_{K_{max}}$$

- Convolution in reduced Fourier feature space

$$(R_\phi)_{K_{max}} \cdot (\mathcal{F}v_t)_{K_{max}}$$

- Inverse Fourier transform to recover the spatial domain

$$\mathcal{F}^{-1}(R_\phi \cdot (\mathcal{F}v_t)) \approx \mathcal{F}^{-1}((R_\phi)_{K_{max}} \cdot (\mathcal{F}v_t)_{K_{max}})$$



Algorithm aspect

- Input $v_t(x_1, \dots, x_K) \in \mathbb{R}^{K \times d_v}$
- for $l \in 1, \dots, d_v$
 - $v_t^l(x_1, \dots, x_K) \in \mathbb{R}^K$ is the l th dimension of v_t
 - d dimensional Fourier transform of $v_t^l(x_1, \dots, x_K)$

$$(\mathcal{F}v_t^l)(k_1, \dots, k_K) \in \mathbb{C}^K$$
 - Approximate $(\mathcal{F}v_t^l)$ with $(\mathcal{F}v_t^l)_{K_{max}} \in \mathbb{C}^{K_{max}}$, where $K_{max} \ll K$
- $(\mathcal{F}v_t)_{K_{max}} \in \mathbb{C}^{K_{max} \times d_v}$
- Convolution in each mode, for mode $k \in 1, \dots, K_{max}$
 - $(\mathcal{F}v_t)_{K_{max}}^k \in \mathbb{C}^{d_v}$, $(R_\phi)_{max}^k \in \mathbb{R}^{d_v \times d_v}$

$$(\mathcal{F}v_t)_{K_{max}}^k \cdot (R_\phi)_{max}^k \in \mathbb{C}^{d_v}$$
- $(\mathcal{F}v_t)_{K_{max}} \cdot (R_\phi) \in \mathbb{C}^{K_{max} \times d_v}$

Example Application: Darcy flow revisited

Darcy flow problem

Given diffusion coefficient field $a(x)$, solve for $u(x)$: $u \approx G_\theta(a)$.

$$\begin{aligned} -\nabla \cdot (a(x) \nabla u(x)) &= f(x), \quad x \in (0, 1)^2 \\ u(x) &= 0, \quad x \in \partial(0, 1)^2 \end{aligned}$$

Applications: the pressure of subsurface flow, the deformation of linearly elastic materials, and the electric potential in conductive materials.

Example Application: Darcy flow revisited

Darcy flow problem

Given diffusion coefficient field $a(x)$, solve for $u(x)$: $u \approx G_\theta(a)$.

$$\begin{aligned} -\nabla \cdot (a(x) \nabla u(x)) &= f(x), \quad x \in (0, 1)^2 \\ u(x) &= 0, \quad x \in \partial(0, 1)^2 \end{aligned}$$

Applications: the pressure of subsurface flow, the deformation of linearly elastic materials, and the electric potential in conductive materials.

Comparison methods:

- NN: Feedforward neural network.
- RBM: Reduced Basis Method (POD)
- FCN: Fully Convolution Networks
- PCANN: PCA based NN
- GNO: Graph neural operator
- MGNO: Multipole GNO
- LNO: Low-rank decomposition + neural operator

Example Application: Darcy flow revisited

Darcy flow problem

Given diffusion coefficient field $a(x)$, solve for $u(x)$: $u \approx G_\theta(a)$.

$$-\nabla \cdot (a(x) \nabla u(x)) = f(x), \quad x \in (0, 1)^2$$

$$u(x) = 0, \quad x \in \partial(0, 1)^2$$

Applications: the pressure of subsurface flow, the deformation of linearly elastic materials, and the electric potential in conductive materials.

Comparison methods:

- NN: Feedforward neural network.
- RBM: Reduced Basis Method (POD)
- FCN: Fully Convolution Networks
- PCANN: PCA based NN
- GNO: Graph neural operator
- MGNO: Multipole GNO
- LNO: Low-rank decomposition + neural operator

Networks	$s = 85$	$s = 141$	$s = 211$	$s = 421$
NN	0.1716	0.1716	0.1716	0.1716
FCN	0.0253	0.0493	0.0727	0.1097
PCANN	0.0299	0.0298	0.0298	0.0299
RBM	0.0244	0.0251	0.0255	0.0259
GNO	0.0346	0.0332	0.0342	0.0369
LNO	0.0520	0.0461	0.0445	—
MGNO	0.0416	0.0428	0.0428	0.0420
FNO	0.0108	0.0109	0.0109	0.0098

Figure: Benchmarks on darcy flow

Physics informed FNO: Physics informed loss function revisit

Consider the PDE parameterized by λ

$$u_t - \mathcal{L}[u; \lambda] = 0, \quad t, x \in [0, T] \times \Omega \quad (12)$$

where $u(x, t)$ denotes the solution, $\mathcal{L}[\cdot; \lambda]$ is a nonlinear operator.

Solving PDE "means" find a solution $u(x, t)$ satisfying (12).

Physics informed FNO: Physics informed loss function revisit

Consider the PDE parameterized by λ

$$u_t - \mathcal{L}[u; \lambda] = 0, \quad t, x \in [0, T] \times \Omega \quad (12)$$

where $u(x, t)$ denotes the solution, $\mathcal{L}[\cdot; \lambda]$ is a nonlinear operator.

Solving PDE "means" find a solution $u(x, t)$ satisfying (12).

Idea: Use NN $h(x, t; \theta)$ to approximate $u(x, t) : u(x, t) \approx h(x, t; \theta)$

- Given λ , the forward problem becomes a optimization problem by minimizing the 'violation' of the governing equation at collocations points $(x_i, t_j), i \in I, j \in J$:

$$\hat{\theta} = \arg \min_{\theta} \sum_{i \in I, j \in J} \|h_t(x_i, t_j) - \mathcal{L}(h(x_i, t_j); \lambda)\|$$

- When you also collected several measurements $u_{obs}(x'_i, t'_j), i \in I', j' \in J'$

$$\hat{\theta} = \arg \min_{\theta} \underbrace{\sum_{i \in I', j' \in J'} \|u_{obs}(x'_i, t'_j) - h(x'_i, t'_j; \lambda)\|}_{\text{data loss}} + \underbrace{\sum_{i \in I, j \in J} \|h_t(x_i, t_j) - \mathcal{L}(h(x_i, t_j); \lambda)\|}_{\text{knowledge loss}}$$

Physics informed FNO: Shall we adopt similar ideas for operator learning?

Darcy flow problem

Given diffusion coefficient field $a(x)$, solve for $u(x)$.

$$-\nabla \cdot (a(x) \nabla u(x)) = f(x), \quad x \in (0, 1)^2 \quad (13)$$

$$u(x) = 0, \quad x \in \partial(0, 1)^2 \quad (14)$$

Solving PDE "means" find a solution $u(x)$ satisfying (13).

Physics informed FNO: Shall we adopt similar ideas for operator learning?

Darcy flow problem

Given diffusion coefficient field $a(x)$, solve for $u(x)$.

$$-\nabla \cdot (a(x) \nabla u(x)) = f(x), \quad x \in (0, 1)^2 \quad (13)$$

$$u(x) = 0, \quad x \in \partial(0, 1)^2 \quad (14)$$

Solving PDE "means" find a solution $u(x)$ satisfying (13).

Idea: Use NN $G_\theta(a)$ to approximate : $G_\theta(a) \approx u$.

- Physics informed loss term:

$$\sum_{j=1}^{J_{dsn}} \lambda_1 \| -\nabla \cdot (a_j^{dsn}(x) \nabla G_\theta(a_j^{dsn})(x)) - f(x) \|_2^2 + \lambda_2 \| G_\theta(a_j^{dsn})(0) \|_2^2$$

where a_j^{dsn} is the 'designed' input.

Physics informed FNO

Loss function

$$\begin{aligned}
 \mathcal{L} = & \underbrace{\sum_{n=1}^N \sum_{k=1}^K \|u_n(x_k) - G_\theta(a_n)(x_k)\|_2^2}_{\text{data loss}} \\
 & + \underbrace{\sum_{j=1}^{J_{dsn}} \lambda_1 \| -\nabla \cdot (a_j^{dsn}(x) \nabla G_\theta(a_j^{dsn})(x)) - f(x) \|_2^2 + \lambda_2 \| G_\theta(a_j^{dsn})(0) \|_2^2}_{\text{physics loss}}
 \end{aligned}$$

Physics informed loss discussion

Aim: learn $G_\theta(a) \approx u$

Physics informed loss term

$$\sum_{j=1}^{J_{dsn}} \lambda_1 \| -\nabla \cdot (a_j^{dsn}(x) \nabla G_\theta(a_j^{dsn})(x)) - f(x) \|_2^2 + \lambda_2 \| G_\theta(a_j^{dsn})(0) \|_2^2$$

Physics informed loss discussion

Aim: learn $G_\theta(a) \approx u$

Physics informed loss term

$$\sum_{j=1}^{J_{dsn}} \lambda_1 \| -\nabla \cdot (a_j^{dsn}(x) \nabla G_\theta(a_j^{dsn})(x)) - f(x) \|_2^2 + \lambda_2 \| G_\theta(a_j^{dsn})(0) \|_2^2$$

- Physics loss term is cheap
 - It provides additional supervision on any design pairs

$$a_j^{dsn} \rightarrow G_\theta(a_j^{dsn})$$

- It can learn the mapping $G_\theta(a) \approx u$, even without experimental (observation) data pairs.

Physics informed loss discussion

Aim: learn $G_\theta(a) \approx u$

Physics informed loss term

$$\sum_{j=1}^{J_{dsn}} \lambda_1 \| -\nabla \cdot (a_j^{dsn}(x) \nabla G_\theta(a_j^{dsn})(x)) - f(x) \|_2^2 + \lambda_2 \| G_\theta(a_j^{dsn})(0) \|_2^2$$

- Physics loss term is cheap
 - It provides additional supervision on any design pairs

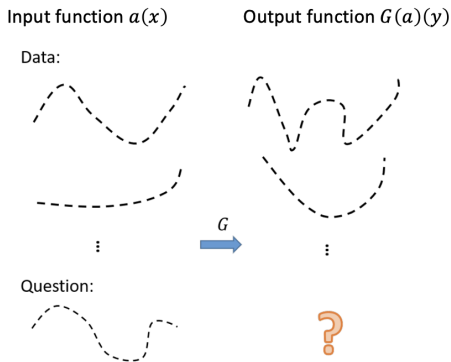
$$a_j^{dsn} \rightarrow G_\theta(a_j^{dsn})$$

- It can learn the mapping $G_\theta(a) \approx u$, even without experimental (observation) data pairs.
- Difficult to balance physics loss term with the data loss term
- **Hard to use automatic differentiation!**

Physics informed operator learning II: DeepOnet

DeepOnet (Lu, Jin, and Karniadakis 2019)

Learning an operator mapping from input function $a(x)$ to output function $u(y) \approx G(a)(y)$.



¹Some of the figures borrow from (https://lululxvi.github.io/files/talks/2020SIAMMS_MS1.pdf)

DeepOnet: Motivation problem

Darcy flow:

$$\begin{aligned} -\nabla \cdot (a(x) \nabla u(x)) &= f(x), \quad x \in (0, 1)^2 \\ u(x) &= 0, \quad x \in \partial(0, 1)^2 \end{aligned}$$

Given diffusion coefficient field $a(x)$, solve for $u(x)$.

DeepOnet: Motivation problem

Darcy flow:

$$\begin{aligned} -\nabla \cdot (a(x) \nabla u(x)) &= f(x), \quad x \in (0, 1)^2 \\ u(x) &= 0, \quad x \in \partial(0, 1)^2 \end{aligned}$$

Given diffusion coefficient field $a(x)$, solve for $u(x)$.

Data: observation pairs a^i, u^i

$$a^i = [a^i(x_1), \dots, a^i(x_m)] \in \mathbb{R}^m, i \in 1, \dots, N$$

$$u^i = [u^i(y_1), \dots, u^i(y_K)] \in \mathbb{R}^K, i \in 1, \dots, N$$

Aim to learn a mapping

$$u^i \approx G_\theta(a^i)$$

DeepOnet Design

Remember we are learning an operator

$$G : a \mapsto G(a)(\cdot)$$

where $G(a)$ is a function

$$G(a)(y) : y \in \mathbb{R}^d \mapsto G(a)(y) \in \mathbb{R}$$

Aim: Given input function a , and any query point location y , output the solution at query point y , i.e., $G(a)(y) \approx u(y)$.

DeepOnet Design

Remember we are learning an operator

$$G : a \mapsto G(a)(\cdot)$$

where $G(a)$ is a function

$$G(a)(y) : y \in \mathbb{R}^d \mapsto G(a)(y) \in \mathbb{R}$$

Aim: Given input function a , and any query point location y , output the solution at query point y , i.e., $G(a)(y) \approx u(y)$.

- Inputs:

- a at 'sensors' $\{x_1, x_2, \dots, x_m\}$,
i.e., $\{a(x_1), a(x_2), \dots, a(x_m)\}$
- y

- Output: $G(a)(y)$

Input: $\{a(x_1), a(x_2), \dots, a(x_m)\}$



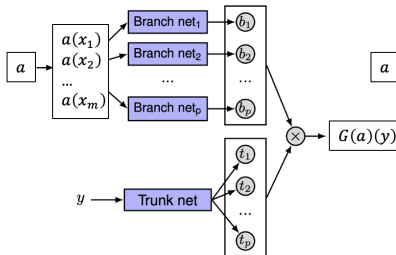
Ideas

Design $G(a)(y)$ as a function of y conditioning on a :

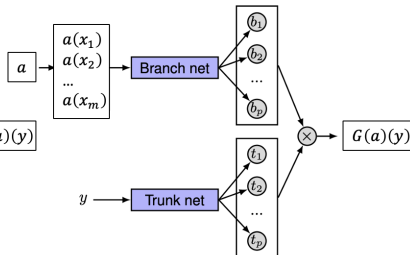
$$G(a)(y) = \sum_{k=1}^p b_k(a) t_k(y)$$

- $t_k(y)$ is the basis function of y
- $b_k(a)$ is the coefficients derived from a

C Stacked DeepONet



D Unstacked DeepONet



Loss function

Loss function

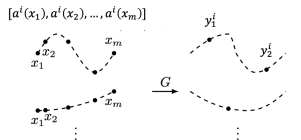
$$\mathcal{L}(\theta) = \sum_{i \in I} \sum_{j \in J_i} \|G_{\theta}(a^i)(y_j^i) - u^i(y_j^i)\|_2^2$$

Input-output data structure:

The i -th data sample

- Input: a^i, y_j^i
- Output: $G(a^i)(y_j^i)$

$$[\mathbf{a}, \mathbf{y}, \mathbf{G}(\mathbf{a})(\mathbf{y})] = \left[\begin{array}{c} a^i(x_1), a^i(x_2), \dots, a^i(x_m), \\ \vdots \\ a^i(x_1), a^i(x_2), \dots, a^i(x_m), \end{array} \right], \left[\begin{array}{c} y_1^i \\ \vdots \\ y_p^i \end{array} \right], \left[\begin{array}{c} G(a^i)(y_1^i) \\ \vdots \\ G(a^i)(y_p^i) \end{array} \right] \right]$$



Physics informed DeepOnet

Loss function

$$\mathcal{L} = \underbrace{\sum_{i \in I} \sum_{j \in J_i} \|G_\theta(a^i)(y_j^i) - u^i(y_j^i)\|_2^2}_{\text{data loss}} + \underbrace{\sum_{i=1}^{I_{dsn}} \lambda_1 \| -\nabla \cdot (a_i^{dsn}(y) \nabla G_\theta(a_i^{dsn}(y))) - f(y) \|_2^2 + \lambda_2 \| G_\theta(a_i^{dsn})(0) \|_2^2}_{\text{physics loss}}$$

Comparison with Physics informed FNO

- Network structure difference:
 - FNO: Input/output high dimensional data
 - PIDeepOnet: Input/output low dimensional data
- Automatic differentiation:
 - FNO: difficult
 - PIDeepOnet: can be directly calculated: $\frac{\partial G_\theta(a)(y)}{\partial y}, \frac{\partial^2 G_\theta(a)(y)}{\partial y^2}$

Table of Contents

- PDEs
- Course Outline

2 PIML for Surrogate modeling

- Introduction to PDE
- Physics informed equation solver
- Physics informed operator learning I: Fourier Neural Operator
- Physics informed operator learning II: DeepOnet

3 PIML for inverse problem

- Identification of PDE governed-system
- Identify unknown boundary/initial conditions
- Identify unknown boundary/initial conditions
- Spatio-temporal modeling of PDE governed-system
- Forward and Inverse problem in a loop

PINN for inverse problem (Raissi, Perdikaris, and Karniadakis 2019)

A simple inverse problem

Consider the PDE parameterized by **unknown** λ

$$u_t - \mathcal{L}[u; \lambda] = 0, \quad t, x \in [0, T] \times \Omega$$

where $u_{obs}(x, t)$ denotes the solution, $\mathcal{L}[\cdot; \lambda]$ is a nonlinear operator.

Problem:

- Given a set of data pairs $[x_i, t_i, u(x_i, t_i)]$, $i \in I$;
- Solve for λ .

PINN for inverse problem (Raissi, Perdikaris, and Karniadakis 2019)

A simple inverse problem

Consider the PDE parameterized by **unknown λ**

$$u_t - \mathcal{L}[u; \lambda] = 0, \quad t, x \in [0, T] \times \Omega$$

where $u_{obs}(x, t)$ denotes the solution, $\mathcal{L}[\cdot; \lambda]$ is a nonlinear operator.

Problem:

- Given a set of data pairs $[x_i, t_i, u(x_i, t_i)]$, $i \in I$;
- Solve for λ .

Solution

- Using NN to approximate $h(x, t; \theta)$
- Minimize two losses:

$$\min_{\theta, \lambda} \underbrace{\sum_{i \in I} \|u_{obs}(x_i, t_i) - h(x_i, t_i; \theta)\|}_{\text{data loss}} + \underbrace{\sum_{i \in I} \|h_t(x_i, t_i; \theta) - \mathcal{L}(h(x_i, t_i; \theta); \lambda)\|}_{\text{PDE loss}}$$

Example

Navier–Stokes equation

Consider the PDE parameterized by **unknown** $\lambda = (\lambda_1, \lambda_2)$

$$\underbrace{u_t + \lambda_1(uu_x + vu_y) - p_x + \lambda_2(u_x x + u_y y)}_f = 0$$

$$\underbrace{v_t + \lambda_1(uu_x + vu_y) - p_y + \lambda_2(v_x x + v_y y)}_g = 0$$

Problem:

- Given a set of data pairs $[x_i, y_i, t_i, u_i, v_i], i \in I$;
- Solve for λ .

Example

Navier–Stokes equation

Consider the PDE parameterized by **unknown** $\lambda = (\lambda_1, \lambda_2)$

$$\underbrace{u_t + \lambda_1(uu_x + vu_y) - p_x + \lambda_2(u_x x + u_y y)}_f = 0$$

$$\underbrace{v_t + \lambda_1(uu_x + vu_y) - p_y + \lambda_2(v_x x + v_y y)}_g = 0$$

Problem:

- Given a set of data pairs $[x_i, y_i, t_i, u_i, v_i], i \in I$;
- Solve for λ .

Solution

- Define a network with two outputs $\psi(t, x, y), p(t, x, y)$ and $u = \psi_y, v = -\psi_x$ (continuity equation for incompressible fluids)

$$\min_{\theta, \lambda} \sum_i (|u(t_i, x_i, y_i) - u_i|^2 + |v(t_i, x_i, y_i) - v_i|^2) + \sum_i (|f(t_i, x_i, y_i)|^2 + |g(t_i, x_i, y_i)|^2)$$

PINN for identifying governing equations (Chen, Liu, and Sun 2021)

An advanced inverse problem

Consider the PDE whose governing equations can be described by a set of nonlinear, coupled, parameterized PDEs in the general

$$u_t + \mathcal{L}[u, u^2, \dots, \nabla u, \nabla^2 u, \nabla u \cdot u, \dots, \lambda] = 0, \quad t, x \in [0, T] \times \Omega$$
$$\mathcal{I}[x \in \Omega, t = 0; u, u_t] = 0 \text{ and } \mathcal{B}[x \in \partial\Omega; u, \nabla u] = 0$$

Problem:

- Given a set of data pairs $\mathcal{D}_u = \{[x_i, t_i, u(x_i, t_i)], i \in I\}$;
- Identify the **unknown governing equation** $\mathcal{L}(\cdot)$.

PINN for identifying governing equations (Chen, Liu, and Sun 2021)

An advanced inverse problem

Consider the PDE whose governing equations can be described by a set of nonlinear, coupled, parameterized PDEs in the general

$$u_t + \mathcal{L}[u, u^2, \dots, \nabla u, \nabla^2 u, \nabla u \cdot u, \dots, \lambda] = 0, \quad t, x \in [0, T] \times \Omega$$

$$\mathcal{I}[x \in \Omega, t = 0; u, u_t] = 0 \text{ and } \mathcal{B}[x \in \partial\Omega; u, \nabla u] = 0$$

Problem:

- Given a set of data pairs $\mathcal{D}_u = \{[x_i, t_i, u(x_i, t_i)], i \in I\}$;
- Identify the **unknown governing equation** $\mathcal{L}(\cdot)$.

Solution

Assumptions:

- Physical law is governed by only a few important terms (which terms unknown)
- Those important terms are covered by a library ϕ of functions

$$\phi = \{1, u, u^2, \dots, u_x, u_y, \dots, u^3 \cdot u_{xy}, \dots\}$$

PINN for identifying governing equations (Chen, Liu, and Sun 2021)

An advanced inverse problem

PDE with unknown governing equation:

$$u_t + \mathcal{L}[u, u^2, \dots, \nabla u, \nabla^2 u, \nabla u \cdot u, \dots, \lambda] = 0, \quad t, x \in [0, T] \times \Omega \dots$$

Problem:

- Given a set of data pairs $\mathcal{D}_u = \{[x_i, t_i, u(x_i, t_i)], i \in I\}$;
- Identify the **unknown governing equation** $\mathcal{L}(\cdot)$.

Solution cont'd

Assumptions:

- Physical law is governed by only a few important terms (which terms unknown)
- Those important terms are covered by a library ϕ of functions

$$\phi = \{1, u, u^2, \dots, u_x, u_y, \dots, u^3 \cdot u_{xy}, \dots\}$$

- $\mathcal{L}$ lives in the space spanned by $\phi = \{1, u, u^2, \dots, u_x, u_y, \dots, u^3 \cdot u_{xy}, \dots\}$

$$\mathcal{L}[u, u^2, \dots, \nabla u, \nabla^2 u, \nabla u \cdot u, \dots, \lambda] \approx \mathbf{\Lambda}^T \phi = [\lambda_1, \lambda_2, \lambda_3, \dots]^T [1, u, u^2, \dots]$$

PINN for identifying governing equations (Chen, Liu, and Sun 2021)

An advanced inverse problem

PDE with unknown governing equation:

$$u_t + \mathcal{L}[u, u^2, \dots, \nabla u, \nabla^2 u, \nabla u \cdot u, \dots, \lambda] = 0, \quad t, x \in [0, T] \times \Omega \dots$$

Problem:

- Given a set of data pairs $\mathcal{D}_u = \{[x_i, t_i, u(x_i, t_i)], i \in I\}$;
- Identify the **unknown governing equation** $\mathcal{L}(\cdot)$.

Solution cont'd

Assumptions:

- Physical law is governed by only a few important terms (which terms unknown)
- Those important terms are covered by a library ϕ of functions

$$\phi = \{1, u, u^2, \dots, u_x, u_y, \dots, u^3 \cdot u_{xy}, \dots\}$$

- $\mathcal{L}$ lives in the space **sparsely** spanned by $\phi = \{1, u, u^2, \dots, u_x, u_y, \dots, u^3 \cdot u_{xy}, \dots\}$.

$$\mathcal{L}[u, u^2, \dots, \nabla u, \nabla^2 u, \nabla u \cdot u, \dots, \lambda] \approx \mathbf{\Lambda}^T \phi = [\lambda_1, \lambda_2, \lambda_3, \dots]^T [1, u, u^2, \dots]$$

PINN for identifying governing equations (Chen, Liu, and Sun 2021)

An advanced inverse problem

PDE with unknown governing equation:

$$u_t + \mathcal{L}[u, u^2, \dots, \nabla u, \nabla^2 u, \nabla u \cdot u, \dots, \lambda] = 0, \quad t, x \in [0, T] \times \Omega \dots$$

Problem:

- Given a set of data pairs $\mathcal{D}_u = \{[x_i, t_i, u(x_i, t_i)], i \in I\}$;
- Identify the **unknown governing equation** $\mathcal{L}(\cdot)$.

A alternative formulation

PDE with unknown governing equation:

$$u_t + [\lambda_1, \lambda_2, \lambda_3, \dots]^T [1, u, u^2, \dots] = 0, \quad t, x \in [0, T] \times \Omega \dots$$

Problem:

- Given a set of data pairs $\mathcal{D}_u = \{[x_i, t_i, u(x_i, t_i)], i \in I\}$;
- Identify the **unknown (sparse) weights** $[\lambda_1, \lambda_2, \lambda_3, \dots]^T$.

PINN for identifying governing equations (Chen, Liu, and Sun 2021)

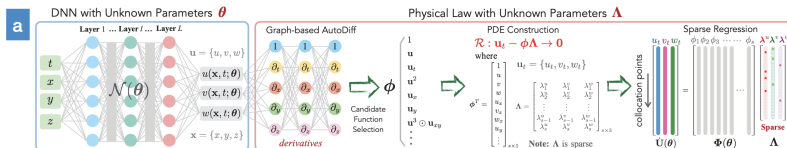
$$u_t + [\lambda_1, \lambda_2, \lambda_3, \dots]^T [1, u, u^2, \dots] = 0, \quad t, x \in [0, T] \times \Omega \dots \quad (15)$$

Problems statement: Given the spatio-temporal measurement data, find a sparse set of Λ such that (15) holds.

Loss function

Define a NN $h(x, t; \theta)$ to approximate $u(x, t)$

$$\min_{\theta, \Lambda} \underbrace{\|h(x_i, t_i; \theta) - u(x_i, t_i)\|_2^2}_{\text{data loss}} + \alpha \underbrace{\|h_t(x_j, t_j; \theta) - \Lambda^T \phi\|_2^2}_{\text{physics loss}} + \beta \underbrace{\|\Lambda\|_0}_{\text{Sparsity}}$$

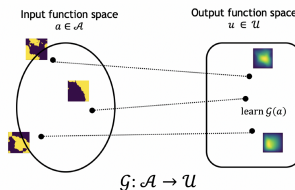


Inversion of Neural Operator (Wang, Wang, and Perdikaris 2021)

Problem statement

Given a set of training data pairs $\mathcal{D} = \{a_i, u_i\}_{i=1}^D$ from the same PDE system with unknown IC/BC:

For any test u^* , find the input a^* .

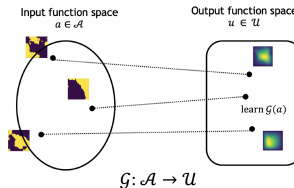


Inversion of Neural Operator (Wang, Wang, and Perdikaris 2021)

Problem statement

Given a set of training data pairs $\mathcal{D} = \{a_i, u_i\}_{i=1}^D$ from the same PDE system with unknown IC/BC:

For any test u^* , find the input a^* .



Parameterization method 1: Forward parameterization

A learned forward model $G_\theta : a \mapsto u$, optimize a by solving:

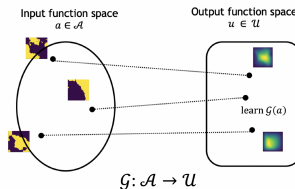
$$\min_a \underbrace{\mathcal{L}_{pde}(a, u^*)}_{\text{PDE (governing Eq. Loss)}} + \underbrace{\mathcal{L}_{data}(G_\theta(a), u^*)}_{\text{forward data loss}} + \underbrace{R(a)}_{\text{regularization}}$$

Inversion of Neural Operator (Wang, Wang, and Perdikaris 2021)

Problem statement

Given a set of training data pairs $\mathcal{D} = \{a_i, u_i\}_{i=1}^D$ from the same PDE system with unknown IC/BC:

For any test u^* , find the input a^* .



Parameterization model 2: Inverse parameterization

A learned inverse model $F_{\theta_0} : u \mapsto a$, fine tune θ by solving:

$$\min_{\theta} \underbrace{\mathcal{L}_{pde}(F_{\theta}(u^*), u^*)}_{\text{PDE (governing Eq. Loss)}} + \underbrace{\mathcal{L}_{op}(F_{\theta_0}, F_{\theta})}_{\text{anchor loss}} + \underbrace{R(F_{\theta})}_{\text{regularization}}$$

Spatio-temporal modeling and prediction

Time-dependent PDE

$$\partial_t u(x, t) = \mathcal{L}u(x, t), \quad t, x \in [0, T] \times \Omega$$

Spatio temporal information is embedded in the equation above.

Spatio-temporal modeling and prediction

Time-dependent PDE

$$\partial_t u(x, t) = \mathcal{L}u(x, t), \quad t, x \in [0, T] \times \Omega$$

Spatio temporal information is embedded in the equation above.

Prerequisites

- Numerical methods: method of lines, PDE-constrained optimization
- Neural networks: Graph convolutional network (GNN)

Spatio-temporal modeling and prediction

Time-dependent PDE

$$\partial_t u(x, t) = \mathcal{L}u(x, t), \quad t, x \in [0, T] \times \Omega$$

Spatio temporal information is embedded in the equation above.

Prerequisites

- Numerical methods: method of lines, PDE-constrained optimization
- Neural networks: Graph convolutional network (GNN)

Expected outcome

- Numerical method inspired approach for integrating physical knowledge (PDE) for spatio-temporal modeling.

Spatio-temporal modeling and prediction

Spatio-temporal modeling and prediction

Observational data: $(\mathbf{y}(t_0), \dots, \mathbf{y}(t_M)) \in \mathbb{R}^{N \times (M+1)}$

- $\mathbf{y}(t) = (u(x_1, t), \dots, u(x_N, t))^T$ is the system state measured at N arbitrary spatial locations $(x_1, \dots, x_N)$ and at $M + 1$ time points $(t_0, \dots, t_M)$.

Spatio-temporal modeling and prediction

Spatio-temporal modeling and prediction

Observational data: $(\mathbf{y}(t_0), \dots, \mathbf{y}(t_M)) \in \mathbb{R}^{N \times (M+1)}$

- $\mathbf{y}(t) = (u(x_1, t), \dots, u(x_N, t))^T$ is the system state measured at N arbitrary spatial locations $(x_1, \dots, x_N)$ and at $M + 1$ time points $(t_0, \dots, t_M)$.

Aim:

- System modeling (identification): $u(x, t) = f(x, t; \theta)$
- (PDE based) Spatio-temporal forecasting: given $\mathbf{y}(t)$ predict $\mathbf{y}(t + i)$, $i > 0$.

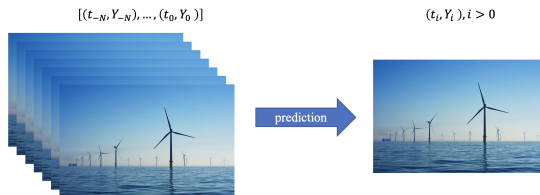


Figure: Wind field forecasting

Spatio-temporal modeling and prediction

Spatio-temporal modeling and prediction

Observational data: $(\mathbf{y}(t_0), \dots, \mathbf{y}(t_M)) \in \mathbb{R}^{N \times (M+1)}$

where

- $\mathbf{y}(t) = (u(x_1, t), \dots, u(x_N, t))^T$ is the system state measured at N arbitrary spatial locations $(x_1, \dots, x_N)$ and at $M + 1$ time points $(t_0, \dots, t_M)$.

Aim:

- System modeling (identification): $u(x, t) = f(x, t; \theta)$
PDE identification with unknown parameters/boundary/initial conditions;
- (PDE based) Spatio-temporal forecasting: given $\mathbf{y}(t)$ predict $\mathbf{y}(t + i)$, $i > 0$.
 1. Physics information incorporated to aid the extrapolation.
 2. **long term** forecasting!

Neural ODE from an optimization perspective

Approximating an ODE

Consider a system (time series model) governed by an Ordinary Differential Equation (ODE):

$$\frac{dh(t)}{dt} = F(h(t), t)$$

where

- F is an unknown function;

Neural ODE from an optimization perspective

Approximating an ODE

Consider a system (time series model) governed by an Ordinary Differential Equation (ODE):

$$\frac{dh(t)}{dt} = F(h(t), t)$$

where

- F is an unknown function;

Question: Given observations at $h(t_0), \dots, h(t_N)$,

- can we estimate the function F ?
- can we make reliable extrapolations for future time points?

An ODE constrained optimization problem

Solution

Approximate F using a neural network f parameterized by θ ; i.e.,

$$\frac{dh(t)}{dt} = f(h(t), t; \theta)$$

An ODE constrained optimization problem

Solution

Approximate F using a neural network f parameterized by θ ; i.e.,

$$\frac{dh(t)}{dt} = f(h(t), t; \theta)$$

Solving the following ODE constrained optimization problem:

$$\min_{\theta} \mathcal{L}(\theta) = \|\hat{h}(t_i) - h(t_i)\|_2^2$$

$$s.t. \quad \hat{h}(t_1), \dots, \hat{h}(t_N) = \text{ODESolve}(f, \theta, h(t_0), t_1, \dots, t_N)$$

An ODE constrained optimization problem

Solution

Approximate F using a neural network f parameterized by θ ; i.e.,

$$\frac{dh(t)}{dt} = f(h(t), t; \theta)$$

Solving the following ODE constrained optimization problem:

$$\begin{aligned} \min_{\theta} \mathcal{L}(\theta) &= \|\hat{h}(t_i) - h(t_i)\|_2^2 \\ \text{s.t. } \hat{h}(t_1), \dots, \hat{h}(t_N) &= \text{ODESolve}(f, \theta, h(t_0), t_1, \dots, t_N) \end{aligned}$$

Once we get the solution for θ , we can

- estimate the dynamics of the system F ;
- facilitate long-term prediction;

An ODE constrained optimization problem

Solution

Approximate F using a neural network f parameterized by θ ; i.e.,

$$\frac{dh(t)}{dt} = f(h(t), t; \theta)$$

Solving the following ODE constrained optimization problem:

$$\begin{aligned} \min_{\theta} \mathcal{L}(\theta) &= \|\hat{h}(t_i) - h(t_i)\|_2^2 \\ \text{s.t. } \hat{h}(t_1), \dots, \hat{h}(t_N) &= \text{ODESolve}(f, \theta, h(t_0), t_1, \dots, t_N) \end{aligned}$$

Once we get the solution for θ , we can

- estimate the dynamics of the system F ;
- facilitate long-term prediction;

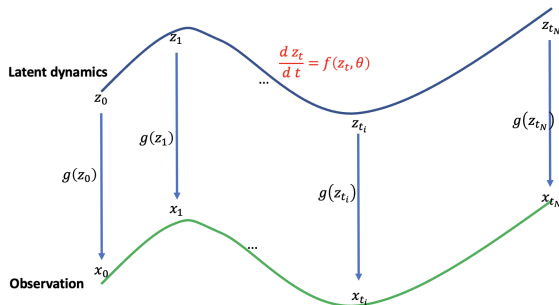
How to solve?

Intuitively:

- For a given θ , we can run ODE solver to get $\hat{h}(t_1), \dots, \hat{h}(t_N)$ and calculate $\mathcal{L}(\theta)$;
- Next, we can calculate the gradient $\frac{\partial \mathcal{L}}{\partial \theta}$ (Adjoint method) [delay to next lecture](#)

Neural ODE (Chen et al. 2018)

From a time series modeling perspective.



Time series modeling:

- Dimensional reduction/ feature extraction modeling: $x_t = g(z_t)$
- Latent dynamics modeling: $z_{t+1} = f(z_t, z_{t-1}, \dots; \theta_f)$

Mathematical model

Time series modeling

Consider the following model:

$$\mathbf{z}_{t_1}, \dots, \mathbf{z}_{t_N} = \text{ODESolve}(\mathbf{z}_{t_0}, f, \theta_f, \dots, t_0, \dots, t_N)$$
$$\mathbf{x}_{t_i} = g(\mathbf{z}_{t_i}; \theta_x)$$

where $\mathbf{x}_{t_i}$ is the observation.

Mathematical model

Time series modeling

Consider the following model:

$$\mathbf{z}_{t_1}, \dots, \mathbf{z}_{t_N} = \text{ODESolve}(\mathbf{z}_{t_0}, f, \theta_f, \dots, t_0, \dots, t_N)$$
$$\mathbf{x}_{t_i} = g(\mathbf{z}_{t_i}; \theta_x)$$

where $\mathbf{x}_{t_i}$ is the observation.

Aim: Given $\{\mathbf{x}_{t_i}, i \in 0, N\}$

- Identify the latent dynamics: find the governing ODE in the latent space f ;
- Identify the mapping from latent space to Observational data g
- Predict future observation points $\mathbf{x}_{t_i}, i = N + 1, \dots$

Mathematical model

Time series modeling

Consider the following model:

$$\begin{aligned} \mathbf{z}_{t_1}, \dots, \mathbf{z}_{t_N} &= \text{ODESolve}(\mathbf{z}_{t_0}, f, \theta_f, \dots, t_0, \dots, t_N) \\ \mathbf{x}_{t_i} &= g(\mathbf{z}_{t_i}; \theta_x) \end{aligned}$$

where $\mathbf{x}_{t_i}$ is the observation.

Aim: Given $\{\mathbf{x}_{t_i}, i \in 0, N\}$

- Identify the latent dynamics: find the governing ODE in the latent space f ;
- Identify the mapping from latent space to Observational data g
- Predict future observation points $\mathbf{x}_{t_i}, i = N + 1, \dots$

Loss function

Solve for θ_f and θ_x :

$$\begin{aligned} \min_{\theta_f, \theta_x} & \quad \|g(\mathbf{z}_{t_i}; \theta_x) - \mathbf{x}_{t_i}\|_2^2 \\ \text{s.t.} \quad & \quad \mathbf{z}_{t_1}, \dots, \mathbf{z}_{t_N} = \text{ODESolve}(\mathbf{z}_{t_0}, f, \theta_f, \dots, t_0, \dots, t_N) \end{aligned}$$

Adjoint method for calculating the derivative

How to solve?

Approximate F using a neural network f parameterized by θ ; i.e.,

$$\frac{dh(t)}{dt} = f(h(t), t; \theta)$$

Solving the following ODE constrained optimization problem:

$$\begin{aligned} \min_{\theta} \mathcal{L}(\theta) &= \|\hat{h}(t_i) - h(t_i)\|_2^2 \\ \text{s.t. } \hat{h}(t_1), \dots, \hat{h}(t_N) &= \text{ODESolve}(f, \theta, h(t_0), t_1, \dots, t_N) \end{aligned}$$

Reading material

- [PDE-constrained optimization and the adjoint method](#)

Adjoint method for ODE constrained optimization problem

A time dependent problem

$$\begin{aligned} \min_{\theta} F(u(\theta), \theta) &= \int_0^T f(u(\theta), \theta, t) dt \\ \text{s.t. } h(u, \dot{u}, \theta, t) &= 0 \\ g(u(0), \theta) &= 0 \end{aligned}$$

where

- $u(\theta)$ is the solution of a ODE, governed by

$$h(u, \dot{u}, \theta, t) = 0$$

with initial condition

$$g(u(0), \theta) = 0$$

- θ is parameter of the ODE
- $f(u(\theta), \theta, t)$ is the loss function, i.e., $\|u(\theta) - u^{obs}\|_2^2$

Adjoint method for ODE constrained optimization problem

A time dependent problem

$$\begin{aligned} \min_{\theta} F(u(\theta), \theta) &= \int_0^T f(u(\theta), \theta, t) dt \\ \text{s.t. } h(u, \dot{u}, \theta, t) &= 0 \\ g(u(0), \theta) &= 0 \end{aligned}$$

where

- $u(\theta)$ is the solution of a ODE, governed by

$$h(u, \dot{u}, \theta, t) = 0$$

with initial condition

$$g(u(0), \theta) = 0$$

- θ is parameter of the ODE
- $f(u(\theta), \theta, t)$ is the loss function, i.e., $\|u(\theta) - u^{obs}\|_2^2$

Aim: find the optimal θ , such that the loss function $F(u(\theta), \theta)$ is minimized.

Adjoint method for ODE constrained optimization problem

Consider the gradient-based optimization (gradient decent method) for unconstrained optimization problem:

$$\theta = \theta - \delta \frac{dF(u, \theta)}{d\theta}$$

Adjoint method for ODE constrained optimization problem

Consider the gradient-based optimization (gradient decent method) for unconstrained optimization problem:

$$\theta = \theta - \delta \frac{dF(u, \theta)}{d\theta}$$

Recall the chain rule

$$\frac{dF(u, \theta, t)}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} \right] dt$$

Adjoint method for ODE constrained optimization problem

Consider the gradient-based optimization (gradient decent method) for unconstrained optimization problem:

$$\theta = \theta - \delta \frac{dF(u, \theta)}{d\theta}$$

Recall the chain rule

$$\frac{dF(u, \theta, t)}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} \right] dt$$

Attention

An example loss function in ML scenario

$$f(u, \theta, t) = \|u(t) - u^{obs}(t)\|_2^2$$

Calculating partial derivatives:

Adjoint method for ODE constrained optimization problem

Consider the gradient-based optimization (gradient decent method) for unconstrained optimization problem:

$$\theta = \theta - \delta \frac{dF(u, \theta)}{d\theta}$$

Recall the chain rule

$$\frac{dF(u, \theta, t)}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} \right] dt$$

Attention

An example loss function in ML scenario

$$f(u, \theta, t) = \|u(t) - u^{obs}(t)\|_2^2$$

Calculating partial derivatives:

- $\frac{\partial f}{\partial u}$ is easy to calculate, i.e., $\frac{\partial f}{\partial u} = 2(u(t) - u^{obs}(t))$

Adjoint method for ODE constrained optimization problem

Consider the gradient-based optimization (gradient decent method) for unconstrained optimization problem:

$$\theta = \theta - \delta \frac{dF(u, \theta)}{d\theta}$$

Recall the chain rule

$$\frac{dF(u, \theta, t)}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} \right] dt$$

Attention

An example loss function in ML scenario

$$f(u, \theta, t) = \|u(t) - u^{obs}(t)\|_2^2$$

Calculating partial derivatives:

- $\frac{\partial f}{\partial u}$ is easy to calculate, i.e., $\frac{\partial f}{\partial u} = 2(u(t) - u^{obs}(t))$
- $\frac{\partial f}{\partial \theta}$ is easy to calculate, i.e., $\frac{\partial f}{\partial \theta} = 0$

Adjoint method for ODE constrained optimization problem

Consider the gradient-based optimization (gradient decent method) for unconstrained optimization problem:

$$\theta = \theta - \delta \frac{dF(u, \theta)}{d\theta}$$

Recall the chain rule

$$\frac{dF(u, \theta, t)}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} \right] dt$$

Attention

An example loss function in ML scenario

$$f(u, \theta, t) = \|u(t) - u^{obs}(t)\|_2^2$$

Calculating partial derivatives:

- $\frac{\partial f}{\partial u}$ is easy to calculate, i.e., $\frac{\partial f}{\partial u} = 2(u(t) - u^{obs}(t))$
- $\frac{\partial f}{\partial \theta}$ is easy to calculate, i.e., $\frac{\partial f}{\partial \theta} = 0$
- $\frac{du}{d\theta}$ is difficult! $\mapsto \{h(u, \dot{u}, \theta, t) = 0; \quad g(u(0), p) = 0\}$

Let's try to avoid the $\frac{du}{d\theta}$ term

A time dependent problem

Consider the following problem:

$$\begin{aligned} \min_{\theta} F(u, \theta) &= \int_0^T f(u, \theta, t) dt \\ \text{s.t. } h(u, \dot{u}, \theta, t) &= 0 \\ g(u(0), \theta) &= 0 \end{aligned}$$

Aim: find the optimal θ , such that the loss function $F(u(\theta), \theta)$ is minimized.

Let's try to avoid the $\frac{du}{d\theta}$ term

A time dependent problem

Consider the following problem:

$$\begin{aligned} \min_{\theta} F(u, \theta) &= \int_0^T f(u, \theta, t) dt \\ \text{s.t. } h(u, \dot{u}, \theta, t) &= 0 \\ g(u(0), \theta) &= 0 \end{aligned}$$

Aim: find the optimal θ , such that the loss function $F(u(\theta), \theta)$ is minimized.

Define the Lagrangian

$$\mathcal{L} = \int_0^T [f(u, \theta, t) + \lambda^T(t) h(u, \dot{u}, \theta, t)] dt + \mu^T g(u(0), \theta)$$

where $\lambda(t)$ and μ are Lagrangian multipliers.

- $\mathcal{L} = F, \forall \lambda(t)$ and μ (Why?) $\Rightarrow \frac{dF}{d\theta} = \frac{d\mathcal{L}}{d\theta}$

Let's calculate $\frac{d\mathcal{L}}{d\theta}$

Lagrangian

$$\mathcal{L} = \int_0^T [f(u, \theta, t) + \lambda^T(t)h(u, \dot{u}, \theta, t)]dt + \mu^T g(u(0), \theta)$$

Gradient

Calculate the gradient $\frac{d\mathcal{L}}{d\theta}$

$$\frac{d\mathcal{L}}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \left(\frac{\partial h}{\partial u} \frac{du}{d\theta} + \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} + \frac{\partial h}{\partial \theta} \right) \right] dt + \mu^T \left(\frac{\partial g}{\partial u(0)} \frac{du(0)}{d\theta} + \frac{\partial g}{\partial \theta} \right)$$

Let's calculate $\frac{d\mathcal{L}}{d\theta}$

Lagrangian

$$\mathcal{L} = \int_0^T [f(u, \theta, t) + \lambda^T(t)h(u, \dot{u}, \theta, t)]dt + \mu^T g(u(0), \theta)$$

Gradient

Calculate the gradient $\frac{d\mathcal{L}}{d\theta}$

$$\frac{d\mathcal{L}}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \left(\frac{\partial h}{\partial u} \frac{du}{d\theta} + \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} + \frac{\partial h}{\partial \theta} \right) \right] dt + \mu^T \left(\frac{\partial g}{\partial u(0)} \frac{du(0)}{d\theta} + \frac{\partial g}{\partial \theta} \right)$$

Eliminate 'hard' terms one-by-one

If we can avoid calculating difficult terms?

- $\frac{du}{d\theta}$
- $\frac{d\dot{u}}{d\theta}$

Eliminate $\frac{d\dot{u}}{d\theta}$

Gradient

Calculate the gradient $\frac{d\mathcal{L}}{d\theta}$

$$\frac{d\mathcal{L}}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \left(\frac{\partial h}{\partial u} \frac{du}{d\theta} + \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} + \frac{\partial h}{\partial \theta} \right) \right] dt + \mu^T \left(\frac{\partial g}{\partial u(0)} \frac{du(0)}{d\theta} + \frac{\partial g}{\partial \theta} \right)$$

Eliminate $\frac{d\dot{u}}{d\theta}$

Gradient

Calculate the gradient $\frac{d\mathcal{L}}{d\theta}$

$$\frac{d\mathcal{L}}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \left(\frac{\partial h}{\partial u} \frac{du}{d\theta} + \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} + \frac{\partial h}{\partial \theta} \right) \right] dt + \mu^T \left(\frac{\partial g}{\partial u(0)} \frac{du(0)}{d\theta} + \frac{\partial g}{\partial \theta} \right)$$

$$\int_0^T \lambda^T \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} dt = \int_0^T \lambda^T \frac{\partial h}{\partial \dot{u}} d \frac{du}{d\theta} \quad (\dot{u} = \frac{du}{dt})$$

Eliminate $\frac{d\dot{u}}{d\theta}$

Gradient

Calculate the gradient $\frac{d\mathcal{L}}{d\theta}$

$$\frac{d\mathcal{L}}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \left(\frac{\partial h}{\partial u} \frac{du}{d\theta} + \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} + \frac{\partial h}{\partial \theta} \right) \right] dt + \mu^T \left(\frac{\partial g}{\partial u(0)} \frac{du(0)}{d\theta} + \frac{\partial g}{\partial \theta} \right)$$

$$\int_0^T \lambda^T \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} dt = \int_0^T \lambda^T \frac{\partial h}{\partial \dot{u}} d \frac{du}{d\theta} \quad (\dot{u} = \frac{du}{dt})$$

recall

$$d(f(x)g(x)) = f(x)d(g(x)) + g(x)d(f(x))$$

Eliminate $\frac{d\dot{u}}{d\theta}$

Gradient

Calculate the gradient $\frac{d\mathcal{L}}{d\theta}$

$$\frac{d\mathcal{L}}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \left(\frac{\partial h}{\partial u} \frac{du}{d\theta} + \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} + \frac{\partial h}{\partial \theta} \right) \right] dt + \mu^T \left(\frac{\partial g}{\partial u(0)} \frac{du(0)}{d\theta} + \frac{\partial g}{\partial \theta} \right)$$

$$\int_0^T \lambda^T \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} dt = \int_0^T \lambda^T \frac{\partial h}{\partial \dot{u}} d \frac{du}{d\theta} \quad (\dot{u} = \frac{du}{dt})$$

recall

$$d(f(x)g(x)) = f(x)d(g(x)) + g(x)d(f(x))$$

then

$$d\left(\lambda^T \frac{\partial h}{\partial \dot{u}} \cdot \frac{du}{d\theta}\right) = \lambda^T \frac{\partial h}{\partial \dot{u}} d \frac{du}{d\theta} + \frac{du}{d\theta} d\left(\lambda^T \frac{\partial h}{\partial \dot{u}}\right)$$

Integrate by parts

$$\begin{aligned}
 & \int_0^T \lambda^T \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} dt \\
 &= \int_0^T \lambda^T \frac{\partial h}{\partial \dot{u}} d \frac{du}{d\theta} \\
 &= \int_0^T d\left(\lambda^T \frac{\partial h}{\partial \dot{u}} \cdot \frac{du}{d\theta}\right) - \frac{du}{d\theta} d\left(\lambda^T \frac{\partial h}{\partial \dot{u}}\right) \\
 &= \lambda^T \frac{\partial h}{\partial \dot{u}} \cdot \frac{du}{d\theta} \Big|_0^T - \int_0^T \frac{du}{d\theta} d\left(\lambda^T \frac{\partial h}{\partial \dot{u}}\right) \\
 &= \lambda^T \frac{\partial h}{\partial \dot{u}} \cdot \frac{du}{d\theta} \Big|_0^T - \int_0^T \frac{du}{d\theta} \left(d\lambda^T \frac{\partial h}{\partial \dot{u}} + \lambda^T d\frac{\partial h}{\partial \dot{u}}\right) \\
 &= \lambda^T \frac{\partial h}{\partial \dot{u}} \cdot \frac{du}{d\theta} \Big|_0^T - \int_0^T \frac{du}{d\theta} \left(\frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} + \lambda^T \frac{d\frac{\partial h}{\partial \dot{u}}}{dt}\right) dt \\
 &= \lambda^T(T) \frac{\partial h}{\partial \dot{u}}(T) \cdot \frac{du}{d\theta}(T) - \lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) \cdot \frac{du}{d\theta}(0) - \int_0^T \frac{du}{d\theta} \left(\frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} + \lambda^T \frac{d\frac{\partial h}{\partial \dot{u}}}{dt}\right) dt
 \end{aligned}$$

Eliminate $\frac{d\dot{u}}{d\theta}$

Gradient

Calculate the gradient $\frac{d\mathcal{L}}{d\theta}$

$$\frac{d\mathcal{L}}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \left(\frac{\partial h}{\partial u} \frac{du}{d\theta} + \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} + \frac{\partial h}{\partial \theta} \right) \right] dt + \mu^T \left(\frac{\partial g}{\partial u(0)} \frac{du(0)}{d\theta} + \frac{\partial g}{\partial \theta} \right)$$

Substitute

$$\begin{aligned} & \int_0^T \lambda^T \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} dt \\ &= \lambda^T(T) \frac{\partial h}{\partial \dot{u}}(T) \cdot \frac{du}{d\theta}(T) - \lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) \cdot \frac{du}{d\theta}(0) - \int_0^T \frac{du}{d\theta} \left(\frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} + \lambda^T \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) dt \end{aligned}$$

Eliminate $\frac{d\dot{u}}{d\theta}$

Gradient

Calculate the gradient $\frac{d\mathcal{L}}{d\theta}$

$$\frac{d\mathcal{L}}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial u} \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \left(\frac{\partial h}{\partial u} \frac{du}{d\theta} + \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} + \frac{\partial h}{\partial \theta} \right) \right] dt + \mu^T \left(\frac{\partial g}{\partial u(0)} \frac{du(0)}{d\theta} + \frac{\partial g}{\partial \theta} \right)$$

Substitute

$$\begin{aligned} & \int_0^T \lambda^T \frac{\partial h}{\partial \dot{u}} \frac{d\dot{u}}{d\theta} dt \\ &= \lambda^T(T) \frac{\partial h}{\partial \dot{u}}(T) \cdot \frac{du}{d\theta}(T) - \lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) \cdot \frac{du}{d\theta}(0) - \int_0^T \frac{du}{d\theta} \left(\frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} + \lambda^T \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) dt \end{aligned}$$

We get

$$\begin{aligned} \frac{d\mathcal{L}}{d\theta} = \int_0^T & \left[\left(\frac{\partial f}{\partial u} + \lambda^T \left(\frac{\partial h}{\partial u} - \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) - \frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} \right) \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \frac{\partial h}{\partial \theta} \right] dt \\ & + \lambda^T(T) \frac{\partial h}{\partial \dot{u}}(T) \cdot \frac{du(T)}{d\theta} + (-\lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) + \mu^T \frac{\partial g}{\partial u(0)}) \frac{du(0)}{d\theta} + \mu^T \frac{\partial g}{\partial \theta} \end{aligned}$$

Eliminate $\frac{du}{d\theta}$

Gradient

The gradient $\frac{d\mathcal{L}}{d\theta}$

$$\begin{aligned} \frac{d\mathcal{L}}{d\theta} = \int_0^T \left[\left(\frac{\partial f}{\partial u} + \lambda^T \left(\frac{\partial h}{\partial u} - \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) - \frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} \right) \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \frac{\partial h}{\partial \theta} \right] dt \\ + \lambda^T(T) \frac{\partial h}{\partial \dot{u}}(T) \cdot \frac{du(T)}{d\theta} + (-\lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) + \mu^T \frac{\partial g}{\partial u(0)}) \frac{du(0)}{d\theta} + \mu^T \frac{\partial g}{\partial \theta} \end{aligned}$$

Recall $\mathcal{L} = F, \forall \lambda(t)$ and μ , we can eliminate $\frac{du}{d\theta}$ by

Eliminate $\frac{du}{d\theta}$

Gradient

The gradient $\frac{d\mathcal{L}}{d\theta}$

$$\begin{aligned} \frac{d\mathcal{L}}{d\theta} = \int_0^T & \left[\left(\frac{\partial f}{\partial u} + \lambda^T \left(\frac{\partial h}{\partial u} - \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) - \frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} \right) \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \frac{\partial h}{\partial \theta} \right] dt \\ & + \lambda^T(T) \frac{\partial h}{\partial \dot{u}}(T) \cdot \frac{du(T)}{d\theta} + (-\lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) + \mu^T \frac{\partial g}{\partial u(0)}) \frac{du(0)}{d\theta} + \mu^T \frac{\partial g}{\partial \theta} \end{aligned}$$

Recall $\mathcal{L} = F, \forall \lambda(t)$ and μ , we can eliminate $\frac{du}{d\theta}$ by

- Set $\lambda(T) = 0 \quad \mapsto \quad \lambda^T(T) \frac{\partial h}{\partial \dot{u}}(T) \cdot \frac{du(T)}{d\theta} = 0$

Eliminate $\frac{du}{d\theta}$

Gradient

The gradient $\frac{d\mathcal{L}}{d\theta}$

$$\begin{aligned} \frac{d\mathcal{L}}{d\theta} = \int_0^T & \left[\left(\frac{\partial f}{\partial u} + \lambda^T \left(\frac{\partial h}{\partial u} - \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) - \frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} \right) \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \frac{\partial h}{\partial \theta} \right] dt \\ & + \lambda^T(T) \frac{\partial h}{\partial \dot{u}}(T) \cdot \frac{du(T)}{d\theta} + (-\lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) + \mu^T \frac{\partial g}{\partial u(0)}) \frac{du(0)}{d\theta} + \mu^T \frac{\partial g}{\partial \theta} \end{aligned}$$

Recall $\mathcal{L} = F, \forall \lambda(t)$ and μ , we can eliminate $\frac{du}{d\theta}$ by

- Set $\lambda(T) = 0 \quad \mapsto \quad \lambda^T(T) \frac{\partial h}{\partial \dot{u}}(T) \cdot \frac{du(T)}{d\theta} = 0$
- Set $\mu^T = \lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) \left(\frac{\partial g}{\partial u(0)} \right)^{-1} \quad \mapsto \quad (-\lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) + \mu^T \frac{\partial g}{\partial u(0)}) \frac{du(0)}{d\theta} = 0$

Eliminate $\frac{du}{d\theta}$

Gradient

The gradient $\frac{d\mathcal{L}}{d\theta}$

$$\begin{aligned} \frac{d\mathcal{L}}{d\theta} = \int_0^T & \left[\left(\frac{\partial f}{\partial u} + \lambda^T \left(\frac{\partial h}{\partial u} - \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) - \frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} \right) \frac{du}{d\theta} + \frac{\partial f}{\partial \theta} + \lambda^T \frac{\partial h}{\partial \theta} \right] dt \\ & + \lambda^T(T) \frac{\partial h}{\partial \dot{u}}(T) \cdot \frac{du(T)}{d\theta} + (-\lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) + \mu^T \frac{\partial g}{\partial u(0)}) \frac{du(0)}{d\theta} + \mu^T \frac{\partial g}{\partial \theta} \end{aligned}$$

Recall $\mathcal{L} = F, \forall \lambda(t)$ and μ , we can eliminate $\frac{du}{d\theta}$ by

- Set $\lambda(T) = 0 \quad \mapsto \quad \lambda^T(T) \frac{\partial h}{\partial \dot{u}}(T) \cdot \frac{du(T)}{d\theta} = 0$
- Set $\mu^T = \lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) \left(\frac{\partial g}{\partial u(0)} \right)^{-1} \quad \mapsto \quad (-\lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) + \mu^T \frac{\partial g}{\partial u(0)}) \frac{du(0)}{d\theta} = 0$
- if we also have

$$\frac{\partial f}{\partial u} + \lambda^T \left(\frac{\partial h}{\partial u} - \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) - \frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} = 0 \quad \forall t > 0$$

Adjoint method

Gradient

The gradient

$$\frac{dF}{d\theta} = \frac{d\mathcal{L}}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial \theta} + \lambda^T \frac{\partial h}{\partial \theta} \right] dt + \lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) \left(\frac{\partial g}{\partial u(0)} \right)^{-1} \frac{\partial g}{\partial \theta}$$

As long as:

Adjoint method

Gradient

The gradient

$$\frac{dF}{d\theta} = \frac{d\mathcal{L}}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial \theta} + \lambda^T \frac{\partial h}{\partial \theta} \right] dt + \lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) \left(\frac{\partial g}{\partial u(0)} \right)^{-1} \frac{\partial g}{\partial \theta}$$

As long as:

The adjoint ODE

A reverse time ODE

$$\frac{\partial f}{\partial u} + \lambda^T \left(\frac{\partial h}{\partial u} - \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) - \frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} = 0 \quad \forall t > 0$$

$$\lambda(T) = 0$$

How to implement

While not converge:

- Calculate for $h(t), \forall t \in [0, T]$ by solving:

$$h(u, \dot{u}, \theta, t) = 0$$

$$g(u(0), \theta) = 0$$

How to implement

While not converge:

- Calculate for $h(t), \forall t \in [0, T]$ by solving:

$$h(u, \dot{u}, \theta, t) = 0$$

$$g(u(0), \theta) = 0$$

- Calculate $\lambda(T), \forall t \in [0, T]$ by solving:

$$\frac{\partial f}{\partial u} + \lambda^T \left(\frac{\partial h}{\partial u} - \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) - \frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} = 0 \quad \forall t > 0$$

$$\lambda(T) = 0$$

How to implement

While not converge:

- Calculate for $h(t), \forall t \in [0, T]$ by solving:

$$\begin{aligned} h(u, \dot{u}, \theta, t) &= 0 \\ g(u(0), \theta) &= 0 \end{aligned}$$

- Calculate $\lambda(T), \forall t \in [0, T]$ by solving:

$$\begin{aligned} \frac{\partial f}{\partial u} + \lambda^T \left(\frac{\partial h}{\partial u} - \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) - \frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} &= 0 \quad \forall t > 0 \\ \lambda(T) &= 0 \end{aligned}$$

- Set

$$\frac{dF}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial \theta} + \lambda^T \frac{\partial h}{\partial \theta} \right] dt + \lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) \left(\frac{\partial g}{\partial u(0)} \right)^{-1} \frac{\partial g}{\partial \theta}$$

How to implement

While not converge:

- Calculate for $h(t), \forall t \in [0, T]$ by solving:

$$\begin{aligned} h(u, \dot{u}, \theta, t) &= 0 \\ g(u(0), \theta) &= 0 \end{aligned}$$

- Calculate $\lambda(T), \forall t \in [0, T]$ by solving:

$$\begin{aligned} \frac{\partial f}{\partial u} + \lambda^T \left(\frac{\partial h}{\partial u} - \frac{d}{dt} \frac{\partial h}{\partial \dot{u}} \right) - \frac{d\lambda^T}{dt} \frac{\partial h}{\partial \dot{u}} &= 0 \quad \forall t > 0 \\ \lambda(T) &= 0 \end{aligned}$$

- Set

$$\frac{dF}{d\theta} = \int_0^T \left[\frac{\partial f}{\partial \theta} + \lambda^T \frac{\partial h}{\partial \theta} \right] dt + \lambda^T(0) \frac{\partial h}{\partial \dot{u}}(0) \left(\frac{\partial g}{\partial u(0)} \right)^{-1} \frac{\partial g}{\partial \theta}$$

- Gradient descent: $\theta \leftarrow \theta - \delta \frac{dF}{d\theta}$

Learning continuous PDE (Iakovlev, Heinonen, and Lähdesmäki 2020)

Governing equation with unknown format

$$u_t = F(x, u, \nabla_x u, \nabla_x^2 u, \dots), \quad x \in \Omega \in \mathbb{R}^d$$

with unknown F (not even unknown parameters!)

Learning continuous PDE (Iakovlev, Heinonen, and Lähdesmäki 2020)

Governing equation with unknown format

$$u_t = \mathbf{F}(x, u, \nabla_x u, \nabla_x^2 u, \dots), \quad x \in \Omega \in \mathbb{R}^d$$

with unknown F (not even unknown parameters!)

Given observational data

$$(\mathbf{y}(t_0), \dots, \mathbf{y}(t_M)) \in \mathbb{R}^{N \times (M+1)}$$

where $\mathbf{y}(t) = (u(x_1, t), \dots, u(x_N, t))^T$ is the system state measured at N arbitrary spatial locations $(x_1, \dots, x_N)$ and at $M + 1$ time points $(t_0, \dots, t_M)$.

Learning continuous PDE (Iakovlev, Heinonen, and Lähdesmäki 2020)

Governing equation with unknown format

$$u_t = \mathbf{F}(x, u, \nabla_x u, \nabla_x^2 u, \dots), \quad x \in \Omega \in \mathbb{R}^d$$

with unknown F (not even unknown parameters!)

Given observational data

$$(\mathbf{y}(t_0), \dots, \mathbf{y}(t_M)) \in \mathbb{R}^{N \times (M+1)}$$

where $\mathbf{y}(t) = (u(x_1, t), \dots, u(x_N, t))^T$ is the system state measured at N arbitrary spatial locations $(x_1, \dots, x_N)$ and at $M + 1$ time points $(t_0, \dots, t_M)$.

Aim

Learn a model from data that can facilitate future prediction, widely used in sound waves, fluid dynamics, heat dissipation, weather patterns, disease progression or cellular kinetics.

MOL: method of lines

Method of lines

Solving partial differential equations (PDEs) in which all but one dimension is discretized (usually the spatial domain). Consider the following one dimensional heat transfer example

$$u_t = u_{xx}$$

MOL: method of lines

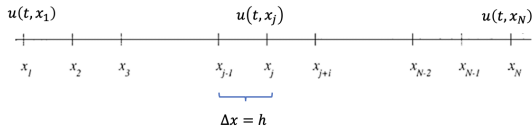
Method of lines

Solving partial differential equations (PDEs) in which all but one dimension is discretized (usually the spatial domain). Consider the following one dimensional heat transfer example

$$u_t = u_{xx}$$

Approximate the spatial derivative by using finite difference scheme:

$$u_{xx}|_{x_j} = \frac{u(x_{j+1}) - 2u(x_j) + u(x_{j-1}))}{h^2}$$



MOL: method of lines

Method of lines

Solving partial differential equations (PDEs) in which all but one dimension is discretized (usually the spatial domain). Consider the following one dimensional heat transfer example

$$u_t = u_{xx}$$

Spatio-temporal PDE becomes a set of ODEs defined on each discretization point x_j :

$$u_t(t, x_j) = \frac{u(t, x_{j+1}) - 2u(t, x_j) + u(t, x_{j-1}))}{h^2}, \quad j \in 1, \dots, N$$

MOL: method of lines

Method of lines: a general case

Assume we want to solve the following PDE with known F :

$$u_t = F(x, u, \nabla_x u, \nabla_x^2 u, \dots), \quad x \in \Omega \subset \mathbb{R}^d$$

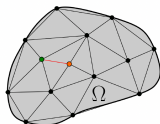


Figure 1: Delaunay triangulation for a set of points. Green and orange points are considered to be neighbors as they share the same edge.

MOL: method of lines

Method of lines: a general case

Assume we want to solve the following PDE with known F :

$$u_t = F(x, u, \nabla_x u, \nabla_x^2 u, \dots), \quad x \in \Omega \subset \mathbb{R}^d$$

- Approximate the spatial derivative by using finite difference:
 - Selecting N nodes in Ω
 - Generate semi-discretized format (spatial discretize and temporal continuous)

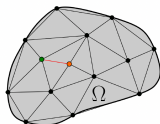


Figure 1: Delaunay triangulation for a set of points. Green and orange points are considered to be neighbors as they share the same edge.

MOL: method of lines

Method of lines: a general case

Assume we want to solve the following PDE with known F :

$$u_t = F(x, u, \nabla_x u, \nabla_x^2 u, \dots), \quad x \in \Omega \subset \mathbb{R}^d$$

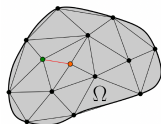


Figure 1: Delaunay triangulation for a set of points. Green and orange points are considered to be neighbors as they share the same edge.

- Approximate the spatial derivative by using finite difference:
 - Selecting N nodes in Ω
 - Generate semi-discretized format (spatial discretize and temporal continuous)
- A set of ODEs

$$u_t(x_i, t) \approx \hat{F}(x_i, x_{\mathcal{N}(i)}, u_i, u_{\mathcal{N}(i)}), \forall i \in 1, \dots, N$$

where

- $\hat{F}$ is discretized F .
- $\mathcal{N}(i)$ is a set of indices of neighboring nodes other than i that are required to evaluate $\hat{F}$ at x_i . (Consider finite element/ finite different discretization!)

MOL: method of lines

Method of lines: a general case

Assume we want to solve the following PDE with known F :

$$u_t = F(x, u, \nabla_x u, \nabla_x^2 u, \dots), \quad x \in \Omega \subset \mathbb{R}^d$$

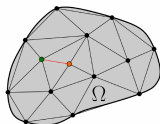


Figure 1: Delaunay triangulation for a set of points. Green and orange points are considered to be neighbors as they share the same edge.

- Approximate the spatial derivative by using finite difference:
 - Selecting N nodes in Ω
 - Generate semi-discretized format (spatial discretize and temporal continuous)
- A set of ODEs

$$u_t(x_i, t) \approx \hat{F}(x_i, x_{\mathcal{N}(i)}, u_i, u_{\mathcal{N}(i)}), \forall i \in 1, \dots, N$$

where

- $\hat{F}$ is discretized F .
- $\mathcal{N}(i)$ is a set of indices of neighboring nodes other than i that are required to evaluate $\hat{F}$ at x_i . (Consider finite element/ finite different discretization!)
- Call ODE solver

Methodology: what if F is unknown?

A thought experiment: there is a ground truth F governing this system, but we do not know it

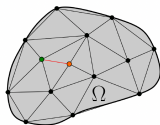


Figure 1: Delaunay triangulation for a set of points. Green and orange points are considered to be neighbors as they share the same edge.

Methodology: what if F is **unknown**?

A thought experiment: there is a ground truth F governing this system, but we do not know it

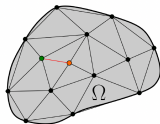


Figure 1: Delaunay triangulation for a set of points. Green and orange points are considered to be neighbors as they share the same edge.

- Apply method of lines:
 - Selecting N nodes in Ω
 - Generate semi-discretized format (spatial discretize and temporal continuous)
- A set of ODEs

$$u_t(x_i, t) \approx \hat{F}(x_i, x_{\mathcal{N}(i)}, u_i, u_{\mathcal{N}(i)}), \forall i \in 1, \dots, N$$

where

- $\hat{F}$ is discretized F , which is till **unknown**.
- $\mathcal{N}(i)$ is a set of indices of neighboring nodes other than i that are required to evaluate $\hat{F}$ at x_i . (Consider finite element/ finite different discretization!)

Methodology: what if F is **unknown**?

A thought experiment: there is a ground truth F governing this system, but we do not know it

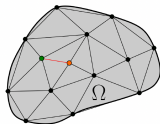


Figure 1: Delaunay triangulation for a set of points. Green and orange points are considered to be neighbors as they share the same edge.

- Apply method of lines:
 - Selecting N nodes in Ω
 - Generate semi-discretized format (spatial discretize and temporal continuous)
- A set of ODEs

$$u_t(x_i, t) \approx \hat{F}(x_i, x_{\mathcal{N}(i)}, u_i, u_{\mathcal{N}(i)}), \forall i \in 1, \dots, N$$

where

- $\hat{F}$ is discretized F , which is till **unknown**.
- $\mathcal{N}(i)$ is a set of indices of neighboring nodes other than i that are required to evaluate $\hat{F}$ at x_i . (Consider finite element/ finite different discretization!)

A set of ODE with **governing equation $\hat{F}$** !

Approximate $\hat{F}$ using GNN

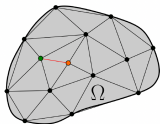


Figure 1: Delaunay triangulation for a set of points. Green and orange points are considered to be neighbors as they share the same edge.

ODE with unknown $\hat{F}$

$$u_t(x_i, t) \approx \hat{F}(x_i, x_{N(i)}, u_i, u_{N(i)}), \forall i \in 1, \dots, N$$

Approximate $\hat{F}$ using GNN

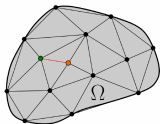


Figure 1: Delaunay triangulation for a set of points. Green and orange points are considered to be neighbors as they share the same edge.

ODE with unknown $\hat{F}$

$$u_t(x_i, t) \approx \hat{F}(x_i, x_{\mathcal{N}(i)}, u_i, u_{\mathcal{N}(i)}), \forall i \in 1, \dots, N$$

Using GNN $\hat{F}_\theta$ to approximate $\hat{F}$

$$u_t(x_i, t) \approx \hat{F}_\theta(x_{\mathcal{N}(i)} - x_i, u_i, u_{\mathcal{N}(i)}) \forall i \in 1, \dots, N$$

where $x_{\mathcal{N}(i)} - x_i = x_j - x_i : j \in \mathcal{N}(i)$.

Assumption: The $\hat{F}$ is position-independent.

Approximate $\hat{F}$ using GNN

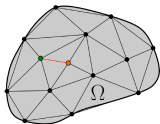


Figure 1: Delaunay triangulation for a set of points. Green and orange points are considered to be neighbors as they share the same edge.

ODE with unknown $\hat{F}$

$$u_t(x_i, t) \approx \hat{F}(x_i, x_{\mathcal{N}(i)}, u_i, u_{\mathcal{N}(i)}), \forall i \in 1, \dots, N$$

Using GNN $\hat{F}_\theta$ to approximate $\hat{F}$

$$u_t(x_i, t) \approx \hat{F}_\theta(x_{\mathcal{N}(i)} - x_i, u_i, u_{\mathcal{N}(i)}) \forall i \in 1, \dots, N$$

where $x_{\mathcal{N}(i)} - x_i = x_j - x_i : j \in \mathcal{N}(i)$.

Assumption: The $\hat{F}$ is position-independent.

Advantage:

- invariant to translations and rotations of Ω ,
- generalization to systems with different node positions, and
- prevents overfitting by memorizing position-specific dynamics.

GNN structure

Consider a K layer GNN with each layer using Message passing GNN (MPGNN).

Message passing layer (MPL)

From the k -th to the $(k + 1)$ -th layer:

Message aggregating:
$$\mathbf{m}_i^{k+1} = \sum_{j \in \mathcal{N}(i)} \phi(k)(\mathbf{h}_i^k, \mathbf{h}_j^k, \mathbf{e}_{i,j})$$

State update:
$$\mathbf{h}_i^{k+1} = \Lambda^k(\mathbf{h}_i^k, \mathbf{m}_i^{k+1})$$

where $\mathbf{e}_{i,j} = \mathbf{x}_j - \mathbf{x}_i$.

GNN structure

Consider a K layer GNN with each layer using Message passing GNN (MPGNN).

Message passing layer (MPL)

From the k -th to the $(k + 1)$ -th layer:

Message aggregating:
$$\mathbf{m}_i^{k+1} = \sum_{j \in \mathcal{N}(i)} \phi(k)(\mathbf{h}_i^k, \mathbf{h}_j^k, \mathbf{e}_{i,j})$$

State update:
$$\mathbf{h}_i^{k+1} = \Lambda^k(\mathbf{h}_i^k, \mathbf{m}_i^{k+1})$$

where $\mathbf{e}_{i,j} = \mathbf{x}_j - \mathbf{x}_i$.

$\hat{F}_\theta$ as GNN

$\hat{F}_\theta(\mathbf{x}_{\mathcal{N}(i)} - \mathbf{x}_i, u_i(t), u_{\mathcal{N}(i)}(t)) :$

- Input: $u_i(t), \mathbf{x}_i$ and the neighbouring point information $u_{\mathcal{N}(i)}(t)$
- MPL iteration
- Output: the approximated first order partial derivative

$$\frac{du_i(t)}{dt} \approx \hat{F}_\theta(\mathbf{x}_{\mathcal{N}(i)} - \mathbf{x}_i, u_i(t), u_{\mathcal{N}(i)}(t))$$

Final model

GNN based PDE surrogate

a set of ODEs with the same functional format

$$u_t(t) = \begin{cases} u_t(x_1, t) \approx \hat{F}_{\theta}(x_{\mathcal{N}(1)} - x_1, u_1, u_{\mathcal{N}(1)}) \\ u_t(x_2, t) \approx \hat{F}_{\theta}(x_{\mathcal{N}(2)} - x_2, u_2, u_{\mathcal{N}(2)}) \\ \dots \\ u_t(x_N, t) \approx \hat{F}_{\theta}(x_{\mathcal{N}(N)} - x_N, u_N, u_{\mathcal{N}(N)}) \end{cases}$$

- Given θ , just regular ODE.
- Predict $(u(x_1, t), \dots, u(x_N, t))^T, \forall t \in 1, \dots, M$ by standard ODE solver (such as Euler and Runge-Kutta solvers).
- Time complexity scales linearly both with respect to the number of nodes N and the number of evaluated time points M .

Loss function and solution method

Loss function

$$\mathcal{L}(\theta) = \int_{t_0}^{t_M} \frac{1}{M+1} \sum_{i=1}^M \|(u(x_1, t_i), \dots, u(x_N, t_i))^T - \mathbf{y}(t_i)\|_2^2 \delta(t - t_i) dt \quad (16)$$

Loss function and solution method

Loss function

$$\mathcal{L}(\theta) = \int_{t_0}^{t_M} \frac{1}{M+1} \sum_{i=1}^M \|(u(x_1, t_i), \dots, u(x_N, t_i))^T - \mathbf{y}(t_i)\|_2^2 \delta(t - t_i) dt \quad (16)$$

Recall Neural ODE

Optimize θ by Adjoint method

Loss function and solution method

Loss function

$$\mathcal{L}(\theta) = \int_{t_0}^{t_M} \frac{1}{M+1} \sum_{i=1}^M \|(u(x_1, t_i), \dots, u(x_N, t_i))^T - \mathbf{y}(t_i)\|_2^2 \delta(t - t_i) dt \quad (16)$$

Recall Neural ODE

Optimize θ by Adjoint method

- **Forward**

- For a given θ , the ODE is known $\hat{F}_\theta$
- Given the initial condition $u(x_j, t_0) = \mathbf{y}(t_0)$, solve the ODE using standard ODE solver
- Get a series prediction results at future time steps $t_1, \dots, t_M$, i.e., $(u(x_1, t_i), \dots, u(x_N, t_i))$;
- Calculate the loss by (16)

Loss function and solution method

Loss function

$$\mathcal{L}(\theta) = \int_{t_0}^{t_M} \frac{1}{M+1} \sum_{i=1}^M \|(u(x_1, t_i), \dots, u(x_N, t_i))^T - \mathbf{y}(t_i)\|_2^2 \delta(t - t_i) dt \quad (16)$$

Recall Neural ODE

Optimize θ by Adjoint method

- **Forward**
 - For a given θ , the ODE is known $\hat{F}_\theta$
 - Given the initial condition $u(x_j, t_0) = \mathbf{y}(t_0)$, solve the ODE using standard ODE solver
 - Get a series prediction results at future time steps $t_1, \dots, t_M$, i.e., $(u(x_1, t_i), \dots, u(x_N, t_i))$;
 - Calculate the loss by (16)
- **Backward**
 - Calculate the gradient with respect to θ , i.e., $(\frac{\partial \mathcal{L}(\theta)}{\partial \theta})$ by using the adjoint method (similar cost as solving the ODE in a reverse manner)

Loss function and solution method

Loss function

$$\mathcal{L}(\theta) = \int_{t_0}^{t_M} \frac{1}{M+1} \sum_{i=1}^M \| (u(x_1, t_i), \dots, u(x_N, t_i))^T - \mathbf{y}(t_i) \|^2 \delta(t - t_i) dt \quad (16)$$

Recall Neural ODE

Optimize θ by Adjoint method

- **Forward**
 - For a given θ , the ODE is known $\hat{F}_\theta$
 - Given the initial condition $u(x_j, t_0) = \mathbf{y}(t_0)$, solve the ODE using standard ODE solver
 - Get a series prediction results at future time steps $t_1, \dots, t_M$, i.e., $(u(x_1, t_i), \dots, u(x_N, t_i))$;
 - Calculate the loss by (16)
- **Backward**
 - Calculate the gradient with respect to θ , i.e., $(\frac{\partial \mathcal{L}(\theta)}{\partial \theta})$ by using the adjoint method (similar cost as solving the ODE in a reverse manner)
- **Update** θ by gradient based solver, i.e., $\theta \rightarrow \theta - l \frac{\partial \mathcal{L}(\theta)}{\partial \theta}$

Experiment: convection-diffusion example

The convection-diffusion equation

$$\begin{aligned}\frac{\partial u(x, y, t)}{\partial t} &= D \nabla^2 u(x, y, t) - \mathbf{v} \cdot \nabla u(x, y, t), & (x, y) \in \Omega, \quad t \geq 0, \\ u(x, 0, t) &= u(x, 2\pi, t), & x \in [0, 2\pi], \quad t \geq 0, \\ u(0, y, t) &= u(2\pi, y, t), & y \in [0, 2\pi], \quad t \geq 0, \\ u(x, y, 0) &= u_0(x, y), & (x, y) \in \Omega,\end{aligned}$$

Initial conditions

$$\begin{aligned}\tilde{u}_0(x, y) &= \sum_{k, l=-N}^N \lambda_{kl} \cos(kx + ly) + \gamma_{kl} \sin(kx + ly) \\ u_0(x, y) &= \frac{\tilde{u}_0(x, y) - \min \tilde{u}_0(x, y)}{\max \tilde{u}_0(x, y) - \min \tilde{u}_0(x, y)},\end{aligned}$$

Varying λ_{kl} and γ_{kl} to generate different initial conditions, each corresponding to one simulation.

Experiment: convection-diffusion example

The convection-diffusion equation

$$\begin{aligned}\frac{\partial u(x, y, t)}{\partial t} &= D \nabla^2 u(x, y, t) - \mathbf{v} \cdot \nabla u(x, y, t), & (x, y) \in \Omega, t \geq 0, \\ u(x, 0, t) &= u(x, 2\pi, t), & x \in [0, 2\pi], t \geq 0, \\ u(0, y, t) &= u(2\pi, y, t), & y \in [0, 2\pi], t \geq 0, \\ u(x, y, 0) &= u_0(x, y), & (x, y) \in \Omega,\end{aligned}$$

Initial conditions

$$\begin{aligned}\tilde{u}_0(x, y) &= \sum_{k, l=-N}^N \lambda_{kl} \cos(kx + ly) + \gamma_{kl} \sin(kx + ly) \\ u_0(x, y) &= \frac{\tilde{u}_0(x, y) - \min \tilde{u}_0(x, y)}{\max \tilde{u}_0(x, y) - \min \tilde{u}_0(x, y)},\end{aligned}$$

Varying λ_{kl} and γ_{kl} to generate different initial conditions, each corresponding to one simulation.

Data

- Training: 24 simulations on the time interval $[0, 0.2]$ sec
- Testing: 50 simulations on the time interval $[0, 0.6]$ sec.

Experiment: Training and test data visualization

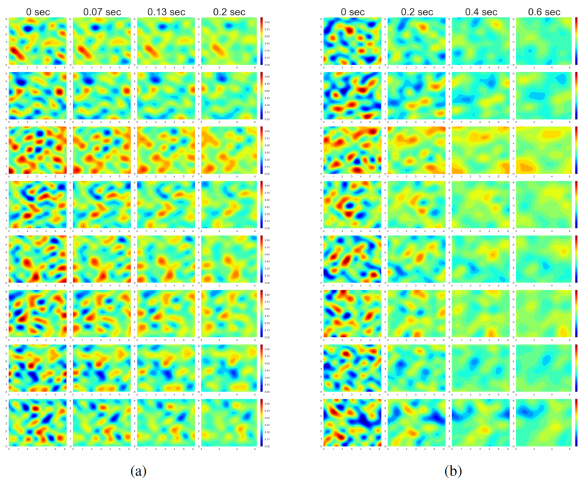


Figure 14: Snapshots of train (a) and test (b) simulations for the convection-diffusion equation.

Experiment results

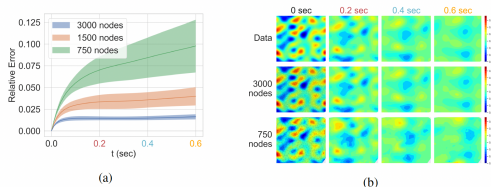


Figure 2: a) Relative test errors for different grid sizes. b) Visualization of the true and learned system dynamics (grids are shown in the first column).

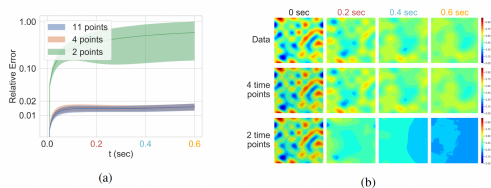


Figure 3: a) Relative test errors for different time grids. b) Visualization of the true and learned system dynamics (grids are shown in the first column).

Data + partial physics knowledge (FEM net (Lienen and Günnemann 2022))

What if we know some additional physics knowledge

Let us assume that the dynamics are not completely unknown, but that a domain expert told us that the generating dynamics of the data include a convective component.

Data + partial physics knowledge (FEM net (Lienen and Günnemann 2022))

What if we know some additional physics knowledge

Let us assume that the dynamics are not completely unknown, but that a domain expert told us that the generating dynamics of the data include a convective component.

- Using finite element method to motivate the approach

Data + partial physics knowledge (FEM net (Lienen and Günnemann 2022))

What if we know some additional physics knowledge

Let us assume that the dynamics are not completely unknown, but that a domain expert told us that the generating dynamics of the data include a convective component.

- Using finite element method to motivate the approach
- Incorporate incomplete physics knowledge through

$$u_t(x_i, t) \approx \hat{F}(x_i, x_{N(i)}, u_i, u_{N(i)}) + F_{known}, \forall i \in 1, \dots, N$$

Data + partial physics knowledge (FEM net (Lienen and Günnemann 2022))

What if we know some additional physics knowledge

Let us assume that the dynamics are not completely unknown, but that a domain expert told us that the generating dynamics of the data include a convective component.

- Using finite element method to motivate the approach
- Incorporate incomplete physics knowledge through

$$u_t(x_i, t) \approx \hat{F}(x_i, x_{N(i)}, u_i, u_{N(i)}) + F_{known}, \forall i \in 1, \dots, N$$

Black sea dataset sea surface temperature and water flow velocities:

- Training: daily data from 01/01/2012 to 31/12/2017
- Validation: 01/01/2018 to 31/12/2018
- Testing: 01/01/2019 to 31/12/2019

CNN and PDE in a loop (Jin et al. 2021)

Full-Waveform Inversion (FWI) [▶ Link](#): Geophysics, Nondestructive testing, Brain imaging, etc: Calculating the wave velocity to characterize subsurface property

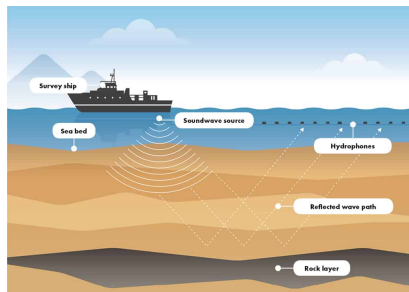


Figure: Full-Waveform Inversion (FWI)⁷

⁶(<https://energyinformationaustralia.com.au/fact-sheets/marine-seismic-surveys-what-you-need-to-know/>)

CNN and PDE in a loop (Jin et al. 2021)

Governing equation

$$\nabla^2 p(\mathbf{r}, t) - \frac{1}{v^2(\mathbf{r})} \frac{\partial^2 p(\mathbf{r}, t)}{\partial t^2} = s(\mathbf{r}, t)$$

where

- $\mathbf{r}$ spatial location, t time
- $p(\mathbf{r}, t)$: pressure field
- $s(\mathbf{r}, t)$: source term (Desinged, Known)
- $v(\mathbf{r}) \in \mathbb{R}^{W \times H}$: velocity map (material parameter, to be inferred)

CNN and PDE in a loop (Jin et al. 2021)

Governing equation

$$\nabla^2 p(\mathbf{r}, t) - \frac{1}{v^2(\mathbf{r})} \frac{\partial^2 p(\mathbf{r}, t)}{\partial t^2} = s(\mathbf{r}, t)$$

where

- $\mathbf{r}$ spatial location, t time
- $p(\mathbf{r}, t)$: pressure field
- $s(\mathbf{r}, t)$: source term (Desinged, Known)
- $v(\mathbf{r}) \in \mathbb{R}^{W \times H}$: velocity map (material parameter, to be inferred)

Problem definition

An inverse problem, given measured $p(\mathbf{r}_i, t_j)$, $i \in [I]$, $j \in [J]$, solve for $v(\mathbf{r})$.

CNN and PDE in a loop (Jin et al. 2021)

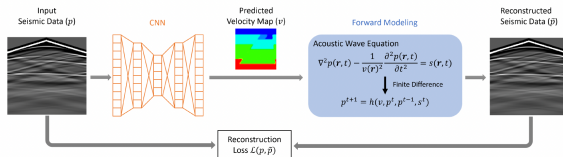


Figure 1: Schematic illustration of our proposed method, which comprises a CNN to learn an inverse mapping and a differentiable operator to approximate the forward modeling of PDE.

- Data: $\mathbf{p} \in \mathbb{R}^{S \times T \times R} = [\mathbf{p}_{t_1}, \dots, \mathbf{p}_{t_T}]$, source, time, receiver

CNN and PDE in a loop (Jin et al. 2021)

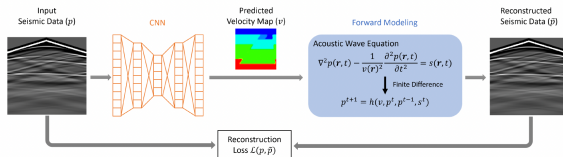


Figure 1: Schematic illustration of our proposed method, which comprises a CNN to learn an inverse mapping and a differentiable operator to approximate the forward modeling of PDE.

- Data: $\mathbf{p} \in \mathbb{R}^{S \times T \times R} = [\mathbf{p}_{t_1}, \dots, \mathbf{p}_{t_T}]$, source, time, receiver
- Forward map using finite difference solver: $f : \mathbb{R}^{W \times H} \mapsto \mathbb{R}^{S \times T \times R}$, i.e., $f(\mathbf{v}) = \mathbf{p}$.

CNN and PDE in a loop (Jin et al. 2021)

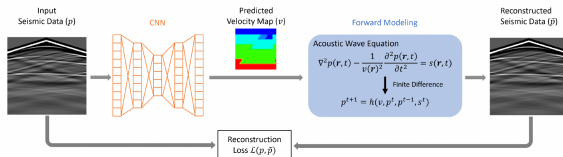


Figure 1: Schematic illustration of our proposed method, which comprises a CNN to learn an inverse mapping and a differentiable operator to approximate the forward modeling of PDE.

- Data: $\mathbf{p} \in \mathbb{R}^{S \times T \times R} = [\mathbf{p}_{t_1}, \dots, \mathbf{p}_{t_T}]$, source, time, receiver
- Forward map using finite difference solver: $f : \mathbb{R}^{W \times H} \mapsto \mathbb{R}^{S \times T \times R}$, i.e., $f(\mathbf{v}) = \mathbf{p}$.
- Inverse map using CNN: $g : \mathbb{R}^{S \times T \times R} \mapsto \mathbb{R}^{W \times H}$, i.e., $g(\mathbf{p}; \theta) = \mathbf{v}$.

CNN and PDE in a loop (Jin et al. 2021)

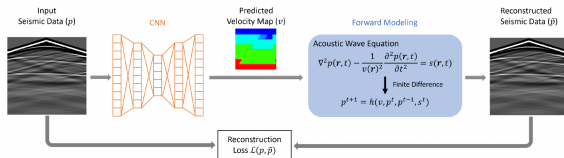


Figure 1: Schematic illustration of our proposed method, which comprises a CNN to learn an inverse mapping and a differentiable operator to approximate the forward modeling of PDE.

- Data: $\mathbf{p} \in \mathbb{R}^{S \times T \times R} = [\mathbf{p}_{t_1}, \dots, \mathbf{p}_{t_T}]$, source, time, receiver
- Forward map using finite difference solver: $f : \mathbb{R}^{W \times H} \mapsto \mathbb{R}^{S \times T \times R}$, i.e., $f(\mathbf{v}) = \mathbf{p}$.
- Inverse map using CNN: $g : \mathbb{R}^{S \times T \times R} \mapsto \mathbb{R}^{W \times H}$, i.e., $g(\mathbf{p}; \theta) = \mathbf{v}$.
- Loss function

$$L(f(g(\mathbf{p}; \theta)) - \mathbf{p}) = L(f(g([\mathbf{p}_{t_1}, \dots, \mathbf{p}_{t_T}]))) - [\mathbf{p}_{t_1}, \dots, \mathbf{p}_{t_T}]$$

CNN and PDE in a loop (Jin et al. 2021)

Differentiable forward finite difference solver:

CNN and PDE in a loop (Jin et al. 2021)

Differentiable forward finite difference solver:

- Second-order central finite difference approximation of $\frac{\partial^2 p(\mathbf{r}, t)}{\partial t^2}$

$$\frac{\partial^2 p(\mathbf{r}, t_i)}{\partial t^2} = \frac{1}{(\Delta t)^2} (p_r^{t_{i+1}} - 2p_r^{t_i} + p_r^{t_{i-1}}) + O[(\Delta t)^2]$$

CNN and PDE in a loop (Jin et al. 2021)

Differentiable forward finite difference solver:

- Second-order central finite difference approximation of $\frac{\partial^2 p(\mathbf{r}, t)}{\partial t^2}$

$$\frac{\partial^2 p(\mathbf{r}, t_i)}{\partial t^2} = \frac{1}{(\Delta t)^2} (p_r^{t_{i+1}} - 2p_r^{t_i} + p_r^{t_{i-1}}) + O[(\Delta t)^2]$$

- Finite difference updating equation

$$p_r^{t_{i+1}} = (2 - v^2 \nabla^2) p_r^{t_i} - p_r^{t_{i-1}} - v^2 (\Delta t)^2 s_r^{t_i}$$

We can solve for $p_r^{t_i}$ at any time point $t_i \in [0, T]$.

CNN and PDE in a loop (Jin et al. 2021)

Differentiable forward finite difference solver:

- Second-order central finite difference approximation of $\frac{\partial^2 p(\mathbf{r}, t)}{\partial t^2}$

$$\frac{\partial^2 p(\mathbf{r}, t_i)}{\partial t^2} = \frac{1}{(\Delta t)^2} (p_r^{t_{i+1}} - 2p_r^{t_i} + p_r^{t_{i-1}}) + O[(\Delta t)^2]$$

- Finite difference updating equation

$$p_r^{t_{i+1}} = (2 - v^2 \nabla^2) p_r^{t_i} - p_r^{t_{i-1}} - v^2 (\Delta t)^2 s_r^{t_i}$$

We can solve for $p_r^{t_i}$ at any time point $t_i \in [0, T]$.

- Gradient of loss can be evaluated as

$$\frac{\partial L}{\partial v(\mathbf{r})} = \sum_{i=1}^T \frac{\partial L}{\partial p(\mathbf{r}, t_i)} \frac{\partial p(\mathbf{r}, t_i)}{\partial v(\mathbf{r})}$$

CNN and PDE in a loop (Jin et al. 2021)

Differentiable forward finite difference solver:

- Second-order central finite difference approximation of $\frac{\partial^2 p(\mathbf{r}, t)}{\partial t^2}$

$$\frac{\partial^2 p(\mathbf{r}, t_i)}{\partial t^2} = \frac{1}{(\Delta t)^2} (p_r^{t_{i+1}} - 2p_r^{t_i} + p_r^{t_{i-1}}) + O[(\Delta t)^2]$$

- Finite difference updating equation

$$p_r^{t_{i+1}} = (2 - v^2 \nabla^2) p_r^{t_i} - p_r^{t_{i-1}} - v^2 (\Delta t)^2 s_r^{t_i}$$

We can solve for $p_r^{t_i}$ at any time point $t_i \in [0, T]$.

- Gradient of loss can be evaluated as

$$\frac{\partial L}{\partial v(\mathbf{r})} = \sum_{i=1}^T \frac{\partial L}{\partial p(\mathbf{r}, t_i)} \frac{\partial p(\mathbf{r}, t_i)}{\partial v(\mathbf{r})}$$

[[SM: Notice this is a long long trajectory, which requires lots of memory to store when calculating this! We can use Deep equilibrium model (or Neural ODE) + finite element net (FEN) to address this issue]]

Differentiable Physics (Thuerey et al. 2021)

A simple example

Let's consider the advection of a passive scalar density $d(\mathbf{x}, t)$ in a velocity field $\mathbf{u}$ as physical model $\mathcal{P}^*$:

$$\frac{\partial d}{\partial t} + \mathbf{u} \cdot \nabla d = 0$$

Differentiable Physics (Thuerey et al. 2021)

A simple example

Let's consider the advection of a passive scalar density $d(\mathbf{x}, t)$ in a velocity field $\mathbf{u}$ as physical model $\mathcal{P}^*$:

$$\frac{\partial d}{\partial t} + \mathbf{u} \cdot \nabla d = 0$$

Aim: an inverse problem

Let's consider the problem of finding a velocity field $\mathbf{u}$. This velocity should transform a given initial scalar density state d^0 at time t^0 into a state that's evolved by $\mathcal{P}$ to a later 'end' time t^e with a certain shape or configuration d^{target} .

$$\min_{\mathbf{u}} L = \|d(t^e) - d^{\text{target}}\|^2$$
$$\frac{\partial d}{\partial t} + \mathbf{u} \cdot \nabla d = 0$$

Differentiable Physics (Thuerey et al. 2021)

A simple example

Let's consider the advection of a passive scalar density $d(\mathbf{x}, t)$ in a velocity field $\mathbf{u}$ as physical model $\mathcal{P}^*$:

$$\frac{\partial d}{\partial t} + \mathbf{u} \cdot \nabla d = 0$$

Aim: an inverse problem

Let's consider the problem of finding a velocity field $\mathbf{u}$. This velocity should transform a given initial scalar density state d^0 at time t^0 into a state that's evolved by $\mathcal{P}$ to a later 'end' time t^e with a certain shape or configuration d^{target} .

$$\min_{\mathbf{u}} L = \|d(t^e) - d^{\text{target}}\|^2$$
$$\frac{\partial d}{\partial t} + \mathbf{u} \cdot \nabla d = 0$$

Can we directly embed the physics equation into the optimization problem?

Differentiable Physics (Thuerey et al. 2021)

After discretization: Let's denote this re-formulation as $\mathcal{P}$. It maps a state of $d(t)$ into a new state at an evolved time, i.e.:

$$d(t + \Delta t) = \mathcal{P}(d(t), \mathbf{u})$$

Differentiable Physics (Thuerey et al. 2021)

After discretization: Let's denote this re-formulation as $\mathcal{P}$. It maps a state of $d(t)$ into a new state at an evolved time, i.e.:

$$d(t + \Delta t) = \mathcal{P}(d(t), \mathbf{u})$$

Assume K step forward evaluation

$$d(t^e) = \mathcal{P}^k(d^0, \mathbf{u})$$

Differentiable Physics (Thuerey et al. 2021)

After discretization: Let's denote this re-formulation as $\mathcal{P}$. It maps a state of $d(t)$ into a new state at an evolved time, i.e.:

$$d(t + \Delta t) = \mathcal{P}(d(t), \mathbf{u})$$

Assume K step forward evaluation

$$d(t^e) = \mathcal{P}^k(d^0, \mathbf{u})$$

Aim: an inverse problem

Let's consider the problem of finding a velocity field $\mathbf{u}$. This velocity should transform a given initial scalar density state d^0 at time t^0 into a state that's evolved by $\mathcal{P}$ to a later 'end' time t^e with a certain shape or configuration d^{target} .

$$\min_{\mathbf{u}} L = |\mathcal{P}^k(d^0, \mathbf{u}) - d^{\text{target}}|^2$$

Differentiable Physics (Thuerey et al. 2021)

After discretization: Let's denote this re-formulation as $\mathcal{P}$. It maps a state of $d(t)$ into a new state at an evolved time, i.e.:

$$d(t + \Delta t) = \mathcal{P}(d(t), \mathbf{u})$$

Assume K step forward evaluation

$$d(t^e) = \mathcal{P}^k(d^0, \mathbf{u})$$

Aim: an inverse problem

Let's consider the problem of finding a velocity field $\mathbf{u}$. This velocity should transform a given initial scalar density state d^0 at time t^0 into a state that's evolved by $\mathcal{P}$ to a later 'end' time t^e with a certain shape or configuration d^{target} .

$$\min_{\mathbf{u}} L = |\mathcal{P}^k(d^0, \mathbf{u}) - d^{\text{target}}|^2$$

The gradient becomes: $\frac{dL}{d\mathbf{u}} = \frac{\partial L}{\partial \mathcal{P}^k(d^0, \mathbf{u})} \frac{d\mathcal{P}^k(d^0, \mathbf{u})}{d\mathbf{u}}$. For simplicity we still use

$$\frac{\partial L}{\partial d(t^e)} \frac{dd(t^e)}{d\mathbf{u}}$$

Differentiate through the physical solver $\mathcal{P}^k(d^0, \mathbf{u})$

First order upwinding scheme

Consider the first order upwinding scheme on a Cartesian grid in 1D, with marker density d_i and velocity u_i for cell i . Discretize the following physics model:

$$\frac{\partial d}{\partial t} + \mathbf{u} \cdot \nabla d = 0$$

Denote as $d_i(t + \Delta t) = \mathcal{P}(d_i(t), \mathbf{u}(t))$, which gives the following:

$$d_i(t + \Delta t) = d_i(t) - \Delta t [u_i^+(d_{i+1}(t) - d_i(t)) + u_i^-(d_i(t) - d_{i-1}(t))] \text{ with}$$

$$u_i^+ = \min(u_i/\Delta x, 0)$$

$$u_i^- = \max(u_i/\Delta x, 0)$$

Differentiate through the physical solver $\mathcal{P}^k(d^0, \mathbf{u})$

First order upwinding scheme

Consider the first order upwinding scheme on a Cartesian grid in 1D, with marker density d_i and velocity u_i for cell i . Discretize the following physics model:

$$\frac{\partial d}{\partial t} + \mathbf{u} \cdot \nabla d = 0$$

Denote as $d_i(t + \Delta t) = \mathcal{P}(d_i(t), \mathbf{u}(t))$, which gives the following:

$$d_i(t + \Delta t) = d_i(t) - \Delta t [u_i^+(d_{i+1}(t) - d_i(t)) + u_i^-(d_i(t) - d_{i-1}(t))] \text{ with}$$

$$u_i^+ = \min(u_i / \Delta x, 0)$$

$$u_i^- = \max(u_i / \Delta x, 0)$$

For a negative u_i we get

$$\mathcal{P}(d_i(t), \mathbf{u}(t)) = (1 + \frac{u_i \Delta t}{\Delta x}) d_i(t) - \frac{u_i \Delta t}{\Delta x} d_{i+1}(t)$$

Differentiate through the physical solver $\mathcal{P}^k(d^0, \mathbf{u})$

For a negative u_i we get

$$\mathcal{P}(d_i(t), \mathbf{u}(t)) = (1 + \frac{u_i \Delta t}{\Delta x}) d_i(t) - \frac{u_i \Delta t}{\Delta x} d_{i+1}(t)$$

Consider a **single step** $\partial \mathcal{P} / \partial d_i$ for cell i from above gives $1 + \frac{u_i \Delta t}{\Delta x}$.

Differentiate through the physical solver $\mathcal{P}^k(d^0, \mathbf{u})$

For a negative u_i we get

$$\mathcal{P}(d_i(t), \mathbf{u}(t)) = (1 + \frac{u_i \Delta t}{\Delta x}) d_i(t) - \frac{u_i \Delta t}{\Delta x} d_{i+1}(t)$$

Consider a **single step** $\partial \mathcal{P} / \partial d_i$ for cell i from above gives $1 + \frac{u_i \Delta t}{\Delta x}$.

The "whole" gradient

$$\frac{dL}{du} = \frac{\partial L}{\partial d(t^e)} \frac{dd(t^e)}{du}$$

Differentiate through the physical solver $\mathcal{P}^k(d^0, \mathbf{u})$

For a negative u_i we get

$$\mathcal{P}(d_i(t), \mathbf{u}(t)) = (1 + \frac{u_i \Delta t}{\Delta x}) d_i(t) - \frac{u_i \Delta t}{\Delta x} d_{i+1}(t)$$

Consider a **single step** $\partial \mathcal{P} / \partial d_i$ for cell i from above gives $1 + \frac{u_i \Delta t}{\Delta x}$.

The "whole" gradient

$$\frac{dL}{du} = \frac{\partial L}{\partial d(t^e)} \frac{dd(t^e)}{du}$$

Chain rule!

$$\frac{dd(t^e)}{du} = \frac{\partial d(t^e)}{\partial \mathbf{u}} + \frac{\partial d(t^e)}{\partial d(t^e - \Delta t)} \frac{dd(t^e - \Delta t)}{du}$$

Differentiate through the physical solver $\mathcal{P}^k(d^0, \mathbf{u})$

For a negative u_i we get

$$\mathcal{P}(d_i(t), \mathbf{u}(t)) = (1 + \frac{u_i \Delta t}{\Delta x}) d_i(t) - \frac{u_i \Delta t}{\Delta x} d_{i+1}(t)$$

Consider a **single step** $\partial \mathcal{P} / \partial d_i$ for cell i from above gives $1 + \frac{u_i \Delta t}{\Delta x}$.

The "whole" gradient

$$\frac{dL}{du} = \frac{\partial L}{\partial d(t^e)} \frac{dd(t^e)}{du}$$

Chain rule!

$$\frac{dd(t^e)}{du} = \frac{\partial d(t^e)}{\partial \mathbf{u}} + \frac{\partial d(t^e)}{\partial (t^e - \Delta t)} \frac{dd(t^e - \Delta t)}{du}$$

$$\frac{dd(t^e - \Delta t)}{du} = \frac{\partial d(t^e - \Delta t)}{\partial \mathbf{u}} + \frac{\partial d(t^e - \Delta t)}{\partial (t^e - 2\Delta t)} \frac{dd(t^e - 2\Delta t)}{du}$$

...

Differentiate through the physical solver $\mathcal{P}^k(d^0, \mathbf{u})$

$$\frac{dL}{d\mathbf{u}} = \frac{\partial L}{\partial d(t^e)} \frac{dd(t^e)}{d\mathbf{u}}$$

$$\frac{dd(t^e)}{d\mathbf{u}} = \frac{\partial d(t^e)}{\partial \mathbf{u}} + \frac{\partial d(t^e)}{\partial d(t^e - \Delta t)} \frac{dd(t^e - \Delta t)}{d\mathbf{u}}$$

$$\frac{dd(t^e - \Delta t)}{d\mathbf{u}} = \frac{\partial d(t^e - \Delta t)}{\partial \mathbf{u}} + \frac{\partial d(t^e - \Delta t)}{\partial d(t^e - 2\Delta t)} \frac{dd(t^e - 2\Delta t)}{d\mathbf{u}}$$

Differentiate through the physical solver $\mathcal{P}^k(d^0, \mathbf{u})$

$$\frac{dL}{d\mathbf{u}} = \frac{\partial L}{\partial d(t^e)} \frac{dd(t^e)}{d\mathbf{u}}$$

$$\frac{dd(t^e)}{d\mathbf{u}} = \frac{\partial d(t^e)}{\partial \mathbf{u}} + \frac{\partial d(t^e)}{\partial d(t^e - \Delta t)} \frac{dd(t^e - \Delta t)}{d\mathbf{u}}$$

$$\frac{dd(t^e - \Delta t)}{d\mathbf{u}} = \frac{\partial d(t^e - \Delta t)}{\partial \mathbf{u}} + \frac{\partial d(t^e - \Delta t)}{\partial d(t^e - 2\Delta t)} \frac{dd(t^e - 2\Delta t)}{d\mathbf{u}}$$

$$\frac{dd(t^e)}{d\mathbf{u}} = \frac{\partial d(t^e)}{\partial \mathbf{u}} + \frac{\partial d(t^e - \Delta t)}{\partial \mathbf{u}} \frac{\partial d(t^e)}{\partial d(t^e - \Delta t)} + \frac{\partial d(t^e)}{\partial d(t^e - \Delta t)} \frac{\partial d(t^e - \Delta t)}{\partial d(t^e - 2\Delta t)} \frac{dd(t^e - 2\Delta t)}{d\mathbf{u}}$$

Differentiate through the physical solver $\mathcal{P}^k(d^0, \mathbf{u})$

$$\frac{dL}{du} = \frac{\partial L}{\partial d(t^e)} \frac{dd(t^e)}{du}$$

$$\begin{aligned} \frac{dL}{d\mathbf{u}} &= \frac{\partial d(t^e)}{\partial \mathbf{u}} \frac{\partial L}{\partial d(t^e)} \\ &+ \frac{\partial d(t^e - \Delta t)}{\partial \mathbf{u}} \frac{\partial d(t^e)}{\partial d(t^e - \Delta t)} \frac{\partial L}{\partial d(t^e)} \\ &+ \dots \\ &+ \left(\frac{\partial d(t^0)}{\partial \mathbf{u}} \dots \frac{\partial d(t^e - \Delta t)}{\partial d(t^e - 2\Delta t)} \frac{\partial d(t^e)}{\partial d(t^e - \Delta t)} \frac{\partial L}{\partial d(t^e)} \right) \end{aligned}$$

Useful Resources

- DeepXDE
- NVIDIA Modulus, A example
- Physics-based Deep Learning Tutorial
- Must-read Papers on Physics-Informed Neural Networks.

Table of Contents

- 1 Introduction and overview of the lecture
 - PDEs
 - Course Outline
- 2 PIML for Surrogate modeling
 - Introduction to PDE
 - Physics informed equation solver
 - Physics informed operator learning I: Fourier Neural Operator
 - Physics informed operator learning II: DeepOnet
- 3 PIML for inverse problem
 - Identification of PDE governed-system
 - Identify unknown boundary/initial conditions
 - Identify unknown boundary/initial conditions
 - Spatio-temporal modeling of PDE governed-system
 - Forward and Inverse problem in a loop
- 4 Summary

Summary I

In this PIML lecture

Summary I

In this PIML lecture

- Forward problem:
 - Physics informed equation solver: PINN
 - Physics informed neural operator: FNO & DeepOnet

Summary I

In this PIML lecture

- Forward problem:
 - Physics informed equation solver: PINN
 - Physics informed neural operator: FNO & DeepOnet
- Inverse Problem

Summary I

In this PIML lecture

- Forward problem:
 - Physics informed equation solver: PINN
 - Physics informed neural operator: FNO & DeepOnet
- Inverse Problem
 - Identify of PDE governed system ([Modeling tools](#))
 - Parameters unknown: PINN inversion
 - Governing equation unknown: PINN inversion + Sparse Regression
 - HD parameters/initial/boundary condition unknown: Neural operator inversion

Summary I

In this PIML lecture

- Forward problem:
 - Physics informed equation solver: PINN
 - Physics informed neural operator: FNO & DeepOnet
- Inverse Problem
 - Identify of PDE governed system ([Modeling tools](#))
 - Parameters unknown: PINN inversion
 - Governing equation unknown: PINN inversion + Sparse Regression
 - HD parameters/initial/boundary condition unknown: Neural operator inversion
 - Spatio-temporal modeling of PDE governed system ([Application 1](#))
 - [Adjoint method](#)
 - Neural ODE
 - Learning continuous PDE from data
 - Finite element network

Summary I

In this PIML lecture

- Forward problem:
 - Physics informed equation solver: PINN
 - Physics informed neural operator: FNO & DeepOnet
- Inverse Problem
 - Identify of PDE governed system (**Modeling tools**)
 - Parameters unknown: PINN inversion
 - Governing equation unknown: PINN inversion + Sparse Regression
 - HD parameters/initial/boundary condition unknown: Neural operator inversion
 - Spatio-temporal modeling of PDE governed system (**Application 1**)
 - **Adjoint method**
 - Neural ODE
 - Learning continuous PDE from data
 - Finite element network
 - Forward and inverse problem in a loop (**Application 2**)
 - **Differentiable physics**
 - CNN + PDE in a loop

Summary II

In this PIML lecture

How to integrate physics information with machine learning method

- Physics constraint as a regularization term: PINN, PIFNO, PIDeepOnet
- Physics knowledge as network structure
 - Rough knowledge: neural network approximate solution $\mapsto$ FNO, DeepOnet.
 - Functional form: neural network approximate governing equation $\mapsto$ NeuralODE, learning continuous PDE, Finite element net
 - Exact solver: Differentiable physics to integrate FEM/FDM solver $\mapsto$ CNN + PDE in a loop

Summary II

In this PIML lecture

How to integrate physics information with machine learning method

- Physics constraint as a regularization term: PINN, PIFNO, PIDeepOnet
- Physics knowledge as network structure
 - Rough knowledge: neural network approximate solution $\mapsto$ FNO, DeepOnet.
 - Functional form: neural network approximate governing equation $\mapsto$ NeuralODE, learning continuous PDE, Finite element net
 - Exact solver: Differentiable physics to integrate FEM/FDM solver $\mapsto$ CNN + PDE in a loop

Numerical method inspired approaches

- Collocation method: PINN
- Finite difference/element: Finite element net, Learning continuous PDE from data, Graph kernel network (FNO)

- 



Raissi, Maziar, Paris Perdikaris, and George E Karniadakis (2019).

“Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. In: *Journal of Computational physics* 378, pp. 686–707.



Thuerey, Nils et al. (2021). *Physics-based Deep Learning*. WWW. URL: <https://physicsbaseddeeplearning.org>.



Wang, Sifan, Hanwen Wang, and Paris Perdikaris (2021). “Learning the solution operator of parametric partial differential equations with physics-informed deepnets”. In: *Science advances* 7.40, eabi8605.